Journal of Computational Physics 569 (2027) 115465

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

Viability of tensor train methods for geophysical fluid dynamics

Jamie Lilly [a,b,*], Derek DeSantis [a], Mark R. Petersen [a]

[a] Computational Physics and Methods Group, Los Alamos National Laboratory, Los Alamos, NM, 87545, USA
[b] Center for Nonlinear Studies, Los Alamos National Laboratory, Los Alamos, NM, 87545, USA

ARTICLE INFO

Keywords:
Tensor train
GPU acceleration
Shallow water
TRiSK
Geophysical fluid dynamics

ABSTRACT

Tensor train (TT) methods have recently gained popularity for accelerating the solving of systems of PDEs. Here, we evaluate the performance of TT methods in the context of geophysical fluid dynamics (GFD) using the shallow water equations and a discretization scheme employed by the ocean component of the Energy Exascale Earth System Model (E3SM). Through a suite of four test cases of increasing complexity, we evaluate TT methods in terms of how much TT is able to compress the model state, the error incurred by the TT approximation, and the speedup obtained by TT versus an optimal standard non-TT implementation in a representative subproblem. We show that though TT is able to effectively compress and speed up simple flows, it struggles to efficiently represent more complex states that are common in realistic GFD applications.

1. Introduction

The central challenge in modeling geophysical fluids is balancing accuracy with computational cost, largely due to the broad range of spatial and temporal scales in play. As spatial resolution is increased to more accurately resolve flow dynamics at smaller scales, the number of unknowns in the numerical system quickly grows, driving up computational expenses. The use of variable-resolution, unstructured grids can help somewhat, allowing geophysical fluid dynamics (GFD) modelers to highly resolve areas of interest while saving on the total number of spatial cells. However, this is often not enough to allow researchers to run the kinds of large ensembles at high resolutions that would accelerate our understanding of vitally important systems like our Earth's climate.

Inspired by new techniques originally developed for quantum physics applications, there has been recent success applying tensor decomposition methods to general computational fluid dynamics (CFD) problems . Given some system of equations, such as that derived from a set of semi-discrete PDEs on a large mesh, one can reformulate the problem in terms of tensor decompositions. Tensor decompositions can discover a low-dimensional latent space to compress high-dimensional interactions, and perform computations there, making previously intractable simulations feasible (see Fig. 1). By far the most popular tensor decomposition method is the so-called tensor train (TT) decomposition wherein a d-dimensional tensor is represented by a "train" of d 3-dimensional tensors [1]; this means that the cost of storing such a tensor scales linearly with d rather than exponentially.

Tensor decomposition methods have been used to model the Navier-Stokes equations in a number of standard test cases: A backward-facing step [2], a lid-driven cavity [3], a turbulent flow in a channel [4], and a 3-dimensional Taylor-Green vortex [5]. Other recent work accelerated compressible flow simulations by up to 1000 times [6] and solved the time-independent Boltzmann neutron transport equation with tensor decomposition methods [7]. These successes show that a tensor-based approach to fluids problems is both possible and could promise greatly increased model efficiency.

* Corresponding author.
E-mail address: jlilly@lanl.gov (J. Lilly).

https://doi.org/10.1016/j.jcp.2026.115465
Received 22 January 2026; Received in revised form 2 July 2026; Accepted 30 September 2026
Available online 10 October 2026
0021-9991/Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

A primary motivation for the current work, a recent study from Danis et al. [8] applies TT methods to the shallow water equations (SWEs) and shows that these methods are capable of accurately resolving certain dynamics in simple GFD problems while providing speedups up to 124x. Seeking to extend these results, we apply TT methods to more realistic applications and compare computational efficiency to that of optimal methods. That is, we seek to test TT methods on more complex GFD flows and on more realistic domains to know how this potential 124x speedup generalizes to realistic problems of interest to the wider community. Pushing toward the climate scale, we consider configurations as close as possible to those used in real-world applications; the particular configuration that is of most interest is that used by the Energy Exascale Earth System Model (E3SM) [9]. The ocean component of E3SM uses the TRiSK spatial discretization [10] which we apply here.

The central question we investigate here (whether TT methods as explored in Danis et al. [8] can supply speedup in more realistic GFD problems) is nontrivial – the acceleration achievable by TT methods is fundamentally conditional. It has been shown that more complicated flows require larger TTs (i.e. less model compression) to capture all relevant features [4]; an open question is whether TT methods provide similar benefits under real-world model configurations like those used in E3SM. For sufficiently complicated flows, the TT may cost more to store in memory than the original "uncompressed" model state. As stated above, the tensor decomposition methods find their origin in the quantum physics space where the tensor train is referred to as a matrix product state (MPS). It can be shown that all realistic quantum states can be efficiently represented as a MPS (i.e. by a tensor train of some high dimensional state tensor) [11]. For this reason, TT is a vital and efficient tool for such applications. In the context of GFD however, we have no such guarantee that any flow state can be efficiently represented by a TT (we discuss this further in Section 2.2.2). Previous work cited above shows that simple flows compress well for a range of CFD applications, but more work is needed to see if TT methods can be generalized to more realistic flows for real-world applications.

In this work, we apply TT methods to the SWEs using an E3SM-like model with the TRiSK spatial discretization. We investigate TT methods as applied to four test cases of increasing complexity, including canonical SWE test cases from Galewsky et al. [12] and Williamson et al. [13]. The TT methods are evaluated based on their ability to compress and accurately represent model states, as well as the speedup they supply versus optimized non-TT methods. We show that performance is strong for simple flows, but that TT methods struggle with the more complex flows from Galewsky et al. [12] and Williamson et al. [13], which implies that TT methods would also struggle with real-world applications at the climate scale.

The paper is structured as follows. In Section 2, we provide the necessary background on tensor trains and how they can be used to solve partial differential equations (PDEs). Next, in Section 3, we describe relevant aspects of the shallow water code used for timing and experiments, as well as issues related to TT on structured and unstructured grids (further discussion of TT on unstructured grids is given in Appendix A). Additionally, we provide details about how we construct sparse operators in the TT-format. Finally, in Section 4, we describe a collection of four test cases and their results, which illustrate the central issue in this work – how effective TT is at compressing flows relevant to GFD.

2. Background

2.1. The shallow water equations

The shallow water equations (SWEs) are a simplification of the Navier-Stokes equations for general fluid flow. The SWEs are obtained from Navier-Stokes under two primary simplifying assumptions; first, that the fluid is of a constant density, and second, that the horizontal length scale of the fluid is much larger than the vertical length scale. The second simplifying assumption has the affect that magnitude of vertical velocities in the fluid become negligible, granting us a two-dimensional flow. Colloquially, this condition can be stated as, *"The Pacific Ocean is shallow. Your coffee cup is deep."* [14].

Though a relatively simple set of governing equations, the SWEs resolve an important subset of physical processes relevant to general geophysical fluid flow [15]. In particular, the SWEs are used to model the so-called barotropic component of full three-dimensional models of the ocean and atmosphere and are therefore often used as a test-bed for new numerical methods targeted at global and climate scale geophysical fluid dynamics. As such, they serve well for our investigation of tensor train methods for this application.

The SWEs on a rotating sphere are given by

$$\begin{aligned} \frac{\partial \mathbf{u}}{\partial t} + (\nabla \times \mathbf{u} + f\mathbf{k}) \times \mathbf{u} &= -\nabla \frac{|\mathbf{u}|^2}{2} - g\nabla(h + z_b) \\ \frac{\partial h}{\partial t} + \nabla \cdot (h\mathbf{u}) &= 0\,, \end{aligned} \tag{1}$$

where $\mathbf{u}(x, y, t) = (u(x, y, t),\ v(x, y, t),\ 0)$ is the two-dimensional fluid velocity, x and y are the spatial coordinates, t is the time coordinate, f is the Coriolis parameter, $\mathbf{k}$ is the local vertical unit vector normal to the sphere, g is the gravitational constant, h is the fluid thickness, and z_b is the height of the bottom topography. We will often refer to the equation for the evolution of $\mathbf{u}$ as the momentum or velocity equation, and the equation for the evolution of h as the thickness or mass equation.

In this work, we solve the SWEs using the TRiSK spatial discretization [10], which is a finite volume-type spatial discretization made for unstructured, variable-resolution polygonal grids, and is the discretization used in the ocean component of E3SM [9]. TRiSK employs C-grid-type discretization [16] wherein the mass variable is computed on cell centers and the normal component of velocity is computed on cell edges. In E3SM, these are Voronoi grids [17,18] consisting primarily of hexagons as the primal mesh, with a dual mesh consisting of triangles. However, TRiSK is not limited to grids of this type; in this work we will primarily utilize quadrilateral grids wherein both the primal and dual grids are quadrilaterals (more on this in Section 3.1).

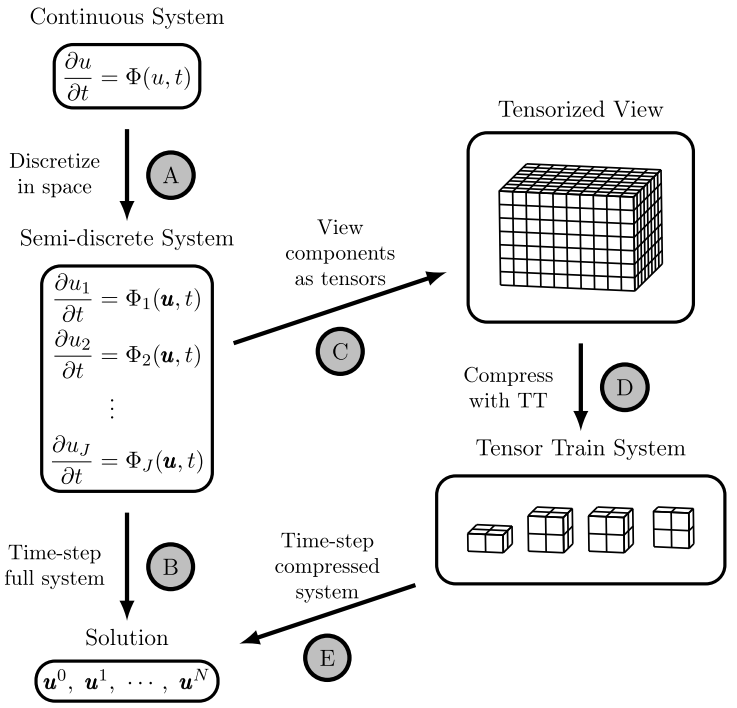


Fig. 1. Tensor train methods find speedups by avoiding time-stepping the full discrete system directly. Standard methods involve (A) discretizing the PDE in space, followed by (B) the use of an iterative time-stepping algorithm. Tensor train methods can be understood to instead (C) treat the discrete system as a collection of tensors, which are (D) approximated by tensor trains. Speedup is found by (E) time-stepping in the reduced dimensional latent space of the tensor trains, which scales linearly with the size of the problem instead of polynomially as in (B).

2.2. Tensor decomposition with tensor train

A tensor decomposition is a method by which a given tensor can be represented by a set of (usually lower dimensional) tensors. This allows high-dimensional data with exponential storage costs to be represented in a more memory-efficient way; one can think of this as obtaining a "compressed" representation of the original tensor. In the present context, a tensor decomposition can be used to formulate a model analogous to a reduced order model (see Example 1 and the discussion at the end of Section 2.2.4). At a high level, the goal of the tensor decomposition methods we investigate here (e.g. tensor train) is to obtain an approximate model state that is significantly compressed versus the true model state, then to evolve the compressed model state forward in time. The idea is that the time evolution of the compressed model state is less computationally expensive at the cost of a controllable amount of error. Fig. 1 provides a high-level summary of this strategy.

However, computational savings that one can obtain with such a tensor decomposition method greatly depends on how well the model state can be compressed. Further, because the model state is changing in time, the compression rate relative to the allowed error will also change in time, potentially greatly affecting the effectiveness a of tensor decomposition method. *This is the central idea that we will investigate in this work.*

2.2.1. Groundwork

To facilitate our discussion of the tensor train (TT) based methods that we use in this work, we begin by carefully defining a number of terms that often appear in different forms across the literature.

Definition 1. A *tensor* is a multi-dimensional array of arbitrary dimension, consisting of real entries, subject to nonlinear point-wise operations and linear transformations. That is, for a positive integer d and a sequence of *mode sizes* $n_1, n_2, \cdots, n_d \in \mathbb{N}$, a tensor $\mathcal{X}$ is an element of $\mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}$. We say that the tensor $\mathcal{X}$ has *shape* $[n_1, n_2, \cdots, n_d]$.

In this way, a $n_1 \times n_2$ matrix is a 2-dimensional tensor in $\mathbb{R}^{n_1 \times n_2}$ and a length n_1 vector is a 1-dimensional tensor in $\mathbb{R}^{n_1}$. A d-dimensional tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}$ is indexed by a multi-index $\mathbf{i} = (i_1, i_2, \cdots, i_d)$, where $i_k \in \{1, 2, \cdots, n_k\}$ for all $k = 1, 2, \cdots, d$; this is written as $\mathcal{X}(i_1, i_2, \cdots, i_d) \in \mathbb{R}$.

Definition 2. A *tensor decomposition* is a representation of a given d-dimensional tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ by some type of product of other tensors. There are many types of products (outer, Khatri-Rao, and Kronecker products) and other multilinear operations used (tensor contractions), which formulate different types of tensor decompositions. Typically the component tensors of a tensor decomposition have dimension less than d.

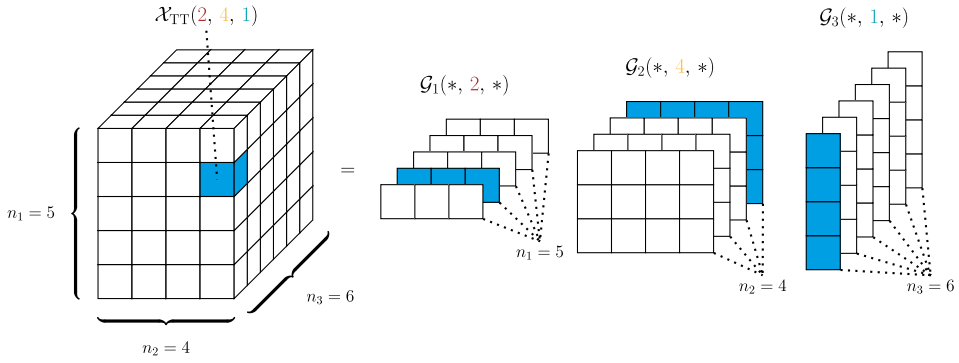


Fig. 2. Visualization of tensor contraction for a tensor train $\mathcal{X}_{\mathrm{TT}} \in \mathbb{R}^{5\times4\times6}$. The entry at multi-index $\mathbf{i} = (2,\ 4,\ 1)$ is obtained by taking the standard matrix-product of the highlighted slices of TT cores $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_3$. This figure was adapted from a similar figure from Xu et al. [22].

The typical example of a tensor decomposition is the tensor train decomposition (Definition 3), but others include canonical polyadic decomposition (CPD) [19], the Tucker decomposition [20], tensor ring [21], and Projected Entangles Parallel Sates (PEPS) [11]. Some tensor decompositions can be viewed as generalizations of matrix decompositions; for example, the singular value decomposition (SVD), LU factorization, or CUR decomposition can be seen as tensor decompositions of 2-dimensional tensors.

Definition 3. A *tensor train* (TT) is a particular tensor decomposition that represents a given d-dimensional tensor $\mathcal{X}_{\mathrm{TT}} \in \mathbb{R}^{n_1\times\cdots\times n_d}$ by a sequence of d 3-dimensional tensors $\mathcal{G}_k$ for $k = 1,\ 2,\ \cdots,\ d$ called *cores*. These cores are sometimes also called *carts*, hence the name tensor "train."

Each core $\mathcal{G}_k$ has shape $[r_{k-1},\ n_k,\ r_k]$, where the $r_k \in \mathbb{N}$ for $k = 0,\ 1,\ \cdots,\ d$, with $r_0 = r_d = 1$, are called the *ranks* of the TT. We often refer to the ranks collectively as a vector quantity $\mathbf{r} = [r_0,\ r_1,\ \cdots,\ r_d]$ called the *rank*. The TT-rank is directly related to how effective a TT representation of a tensor is – the smaller the TT-ranks, the smaller the TT-cores, and so the fewer elements stored by the TT in total (e.g. see Remark 1 and Definition 4).

The TT representation of the tensor $\mathcal{X}_{\mathrm{TT}}$ can be written as

$$\mathcal{X}_{\mathrm{TT}}(i_1,\ \cdots,\ i_d) = \sum_{\alpha_1,\cdots,\alpha_{d-1}}^{r_1,\cdots r_{d-1}} \mathcal{G}_1(1,\ i_1,\ \alpha_1)\ \mathcal{G}_2(\alpha_1,\ i_2,\ \alpha_2)\ \cdots \\ \mathcal{G}_{d-1}(\alpha_{d-2},\ i_{d-1},\ \alpha_{d-1})\ \mathcal{G}_d(\alpha_{d-1},\ i_d,\ 1)\,. \tag{2}$$

Equivalently, we can also denote the TT-format by the multiple matrix product,

$$\mathcal{X}_{\mathrm{TT}}(i_1,\ i_2,\ \ldots,\ i_d) = \mathbf{G}_1(i_1)\ \mathbf{G}_2(i_2)\ \cdots\ \mathbf{G}_d(i_d)\,, \tag{3}$$

where each term $\mathbf{G}_k(i_k)$, for $i_k = 1,\ \cdots,\ n_k$ and $k = 1,\ \cdots, d$, is a matrix of shape $[r_{k-1},\ r_k]$.

These statements can be nontrivial to understand at first glance, so we direct to reader's attention to Fig. 2, which provides a visual example of the tensor contraction in Eqs. (2) and (3).

2.2.2. Approximating tensors by tensor trains

In the present context (solving systems of PDEs), it is most common to use tensor train to approximate a given tensor, e.g. (D) in Fig. 1. Given some tensor $\mathcal{X} \in \mathbb{R}^{n_1\times\cdots\times n_d}$ and some $\varepsilon > 0$, there exists a tensor train $\mathcal{X}_{\mathrm{TT}}$ such that

$$\|\mathcal{X} - \mathcal{X}_{\mathrm{TT}}\|_{\mathrm{F}} < \varepsilon\|\mathcal{X}\|_{\mathrm{F}}\,, \tag{4}$$

where $\|\cdot\|_{\mathrm{F}}$ is the Frobenius norm. In practice, there are a number of ways that $\mathcal{X}_{\mathrm{TT}}$ can be obtained from a particular $\mathcal{X}$ and ε, but the most common is via the so-called TT-SVD algorithm from Oseledets [1, Algorithm 1]. This is a well-known algorithm, so we will not reproduce it in full here. In summary, TT-SVD produces a TT by taking a total of d truncated SVDs of certain unfoldings of the tensor $\mathcal{X}$. For each $k = 1,\ \cdots,\ d$, the kth application of the truncated SVD produces the core $\mathcal{G}_k$ and rank r_k, where r_k is exactly the number of singular values left over after the singular value truncation. As an example, in the simple case where $d = 2$, a TT is obtained from applying a single SVD to the 2-dimensional input tensor (a matrix), see Fig. 3.

Let $n = \max_k\{n_k\}$ and $r = \max_k\{r_k\}$, then the cost to store all of tensor $\mathcal{X}$ is $\mathcal{O}(n^d)$, while the cost to store $\mathcal{X}_{\mathrm{TT}}$ is only $\mathcal{O}(ndr^2)$ – linear in both n and d. For sufficiently controlled values of r, this means that $\mathcal{X}_{\mathrm{TT}}$ is both more efficient to store and more efficient to operate on, e.g. (E) in Fig. 1. The difficulty, which is the primary concern of this paper, is that the TT-rank $\mathbf{r}$ is a function of the desired accuracy ε. When ε is taken to be small, more singular values are kept during each step of the TT-SVD algorithm, and the ranks r_k grow appropriately. Further, the TT-rank $\mathbf{r}$ also depends on $\mathcal{X}$ – if the singular values of the unfoldings of $\mathcal{X}$ decay slowly, more singular values and vectors need to be kept, again resulting in higher TT-rank.

The TT-format was popularized in the quantum materials space, where it is referred to as the Matrix Product State (MPS). In this context, a tensor (usually of large dimension) is used to represent some quantum state, and it can be shown that the set of states that

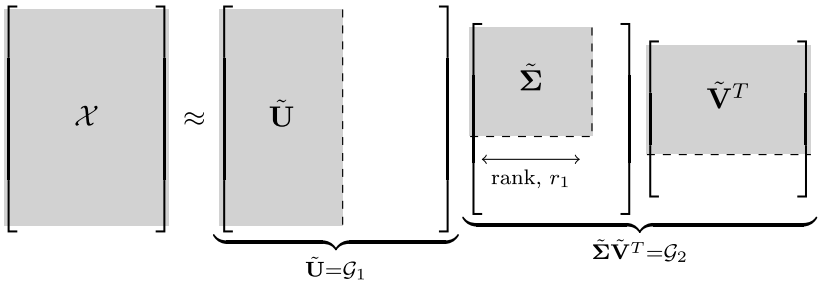


Fig. 3. An example of a tensor train of a 2-dimensional tensor $\mathcal{X}$ (a matrix). A single truncated SVD is applied to $\mathcal{X}$, resulting in the cores $\mathcal{G}_1 = \tilde{\mathbf{U}}$ and $\mathcal{G}_2 = \tilde{\mathbf{\Sigma}}\tilde{\mathbf{V}}^T$. The TT rank in this case is $\mathbf{r} = [1, r_1, 1]$, where r_1 is exactly the number of singular values kept after the SVD truncation. For d-dimensional tensors, the process is similar; a total of $d-1$ truncated SVDs are applied to certain unfoldings of the input tensor, and the component matrices for each SVD are reshaped and permuted appropriately.

are relevant to the study of quantum materials (so-called "Area-law" states) can be efficiently represented as MPSs [11]. In general however, as in our case, there is no guarantee that an arbitrary tensor can be approximated by a TT of low rank.

Remark 1. Given any tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$, there exists a sequence of TTs $\left\{ \mathcal{X}_{\mathrm{TT}}^{(n)} \right\}_{n=1}^{\infty} \subset \mathbb{R}^{n_1 \times \cdots \times n_d}$ such that

$$\|\mathcal{X} - \mathcal{X}_{\mathrm{TT}}^{(n)}\| \to 0 \quad \text{as} \quad n \to \infty\,. \tag{5}$$

However, the TT rank of $\mathcal{X}_{\mathrm{TT}}^{(n)}$ must grow to cover $\mathbb{R}^{n_1 \times \cdots \times n_d}$ [11], so there exists $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ such that Eq. (5) holds while

$$\max_k \left\{ r_k^{(n)} \right\} \to D \quad \text{as} \quad n \to \infty\,, \tag{6}$$

where $r_k^{(n)}$ for $k = 0, \cdots, d$ are the ranks of $\mathcal{X}_{\mathrm{TT}}^{(n)}$, and D is the dimension of $\mathbb{R}^{n_1 \times \cdots \times n_d}$. This statement can be generalized to arbitrary Hilbert spaces $\mathbb{H}$ and $D = \infty$, see Orús [11]. In such a context, it can be said that the set of elements of $\mathbb{H}$ that can be represented by TTs are *dense* in $\mathbb{H}$.

The consequence of Remark 1 in the present context is that while we can approximate any given model state by a TT to arbitrary accuracy, it may be that the TT-rank is unacceptably high, to the point that it is more expensive to store the TT approximation than the original full tensor. To reiterate, the difference between our application and that of quantum materials is that in the quantum materials context, it can be shown that a realistic given state can be represented efficiently as a TT, whereas here we have no such guarantee. The purpose of this work is to investigate this TT compression efficiency for GFD flows with empirical experiments.

Definition 4. The tensor train *compression* rate is simply the ratio between the number of elements (e.g. individual floating point numbers) stored by a full tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ and its TT approximation $\mathcal{X}_{\mathrm{TT}}$,

$$\mathrm{comp}(\mathcal{X}, \mathcal{X}_{\mathrm{TT}}) = \frac{\mathrm{numel}(\mathcal{X}_{\mathrm{TT}})}{\mathrm{numel}(\mathcal{X})}\,, \tag{7}$$

where $\mathrm{numel}(\cdot)$ denotes the total number of elements stored. From the discussion in the second paragraph of Section 2.2.2, we know that

$$\mathrm{comp}(\mathcal{X}, \mathcal{X}_{\mathrm{TT}}) \sim \mathcal{O}\left(\frac{dr^2}{n^{d-1}} \right). \tag{8}$$

For computational performance, we want $\mathrm{comp}(\mathcal{X}, \mathcal{X}_{\mathrm{TT}})$ to be as small as possible while maintaining an acceptable error $\|\mathcal{X} - \mathcal{X}_{\mathrm{TT}}\|_{\mathrm{F}}$.

2.2.3. Operating on Tensor Trains

Tensor trains are, similar to vectors, subject to nonlinear point-wise operations and linear transformations. That is, we can point-wise multiply and add TTs, and also subject TTs to operations analogous to matrix-vector multiplications via tensor train operators.

Definition 5. A *tensor train operator* (TTO) is a linear operator $L_{\mathrm{TTO}} : \mathbb{R}^{n_1 \times \cdots \times n_d} \to \mathbb{R}^{m_1 \times \cdots \times m_d}$ that maps tensor trains to tensor trains. These can be represented as tensors $\mathcal{L}_{\mathrm{TTO}} \in \mathbb{R}^{m_1 \times n_1 \times m_2 \times n_2 \times \cdots \times m_d \times n_d}$, which in turn are written in a tensor train format similar to that defined in Definition 3, but with 4-dimensional cores of shape $[r_{k-1}, m_k, n_k, r_k]$ for $k = 1, \cdots, d$, with $r_0 = r_d = 1$.

A TTO $\mathcal{L}_{\mathrm{TTO}}$ acts on a TT, $\mathcal{X}_{\mathrm{TT}} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ on a "core-by-core" basis. If we denote the cores of $\mathcal{L}_{\mathrm{TTO}}$ by $\mathcal{F}_k \in \mathbb{R}^{r_{k-1} \times m_k \times n_k \times r_k}$ and the cores of $\mathcal{X}_{\mathrm{TT}}$ by $\mathcal{G}_k \in \mathbb{R}^{\tilde{r}_{k-1} \times n_k \times \tilde{r}_k}$, then for each k the tensor $\mathcal{F}_k$ is contracted with $\mathcal{G}_k$ along their axis of shared length n_k to produce a new core $\mathcal{H}_k \in \mathbb{R}^{r_{k-1}\tilde{r}_{k-1} \times m_k \times r_k \tilde{r}_k}$. Together, the new cores form a new TT, $\mathcal{Y}_{\mathrm{TT}} \in \mathbb{R}^{m_1 \times \cdots \times m_d}$. For legibility, we write this operation as $\mathcal{L}_{\mathrm{TTO}}\mathcal{X}_{\mathrm{TT}} = \mathcal{Y}_{\mathrm{TT}}$.

An important property of the TT-format is that standard numerical operations between TTs return a TT. This means that (E) from Fig. 1 can take place entirely in the TT format, without ever forming the full tensor of the model state. However, the rank of the TT returned by such an operation depends on the rank of the operands as shown in Table 1. To combat rank growth during series of TT operations, a type of re-approximation algorithm needs to be applied.

Table 1
The ranks of resultants from TT operations for $\mathcal{X}_{\text{TT}}, \mathcal{Y}_{\text{TT}} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ and $\mathcal{L}_{\text{TTO}} \in \mathbb{R}^{m_1 \times n_1 \times m_2 \times n_2 \times \cdots \times m_d \times n_d}$ with ranks $\mathbf{r}_x$, $\mathbf{r}_y$ and $\mathbf{r}_\ell$ respectively. Here, $*$ denotes an element-wise product.

Operation	Rank of result
$\mathcal{X}_{\text{TT}} + \mathcal{Y}_{\text{TT}}$	$\mathbf{r}_x + \mathbf{r}_y$
$\mathcal{X}_{\text{TT}} * \mathcal{Y}_{\text{TT}}$	$\mathbf{r}_x * \mathbf{r}_y$
$\mathcal{L}_{\text{TTO}} \mathcal{X}_{\text{TT}}$	$\mathbf{r}_\ell * \mathbf{r}_x$

Definition 6. A TT *rounding* algorithm (also called a re-truncation algorithm) is a method that approximates a given TT by a new TT of lower rank. For a TT, $\mathcal{X}_{\text{TT}}$ with ranks $\mathbf{r}_x$ and $\varepsilon > 0$, this is written as

$$\mathcal{Y}_{\text{TT}} = \text{round}(\mathcal{X}_{\text{TT}}, \varepsilon), \tag{9}$$

where $\mathcal{Y}_{\text{TT}}$ has ranks $\mathbf{r}_y$, $\|\mathcal{X}_{\text{TT}} - \mathcal{Y}_{\text{TT}}\|_{\text{F}} < \varepsilon$, and $\mathbf{r}_y \leq \mathbf{r}_x$. The cost of TT-rounding is nontrivial and depends on the rank of the input TT; $\mathcal{O}(dnr^6)$ [1].

Often it is possible to find $\mathcal{Y}_{\text{TT}}$ such that the rank $\mathbf{r}_y$ is much smaller than $\mathbf{r}_x$. The benefit of a rounding algorithm rather than computing a new TT from scratch is that there is no need to first form the full tensor from $\mathcal{X}_{\text{TT}}$.

The most common TT-rounding algorithm is given in Oseledets [1, Algorithm 2], which re-compresses an input TT by first orthogonalizing its cores via a series of QR decompositions, then compresses those cores with a series of truncated SVDs. During a time-integration procedure (e.g. (E) in Fig. 1), it is common practice to perform a TT-round on the state tensor after each time-step, or even after each substage of a given time-stepping method.

2.2.4. Solving PDEs with tensor train

Here, we provide a example of using TT to solve a simple PDE. The example uses the background introduced in this section and serves to illustrate more of the details eluded to by Fig. 1. In this example, we only need to apply linear operations in the TT-format, but in general, more complex nonlinear operations can also be performed in the TT-format via so-called "cross approximation" algorithms, see e.g. Danis et al. [8], Savostyanov and Oseledets [23].

Example 1. Let u be some quantity of interest and consider the simple advection of u in one spatial dimension,

$$\frac{\partial u}{\partial t} = -\frac{\partial u}{\partial x}. \tag{10}$$

We discretize this equation in space via some given finite-difference or finite-volume method and write the resulting problem as

$$\frac{\partial \mathbf{u}}{\partial t} = -\mathbf{D}\mathbf{u}, \tag{11}$$

where $\mathbf{u}$ is a vector giving the values of u at a number of discrete grid points and $\mathbf{D}$ is a matrix encoding a given discrete form of $\partial/\partial x$. Moving from Eq. (10) to Eq. (11) corresponds to (A) in Fig. 1.

Then, both $\mathbf{D}$ and $\mathbf{u}$ can be represented by tensor trains; $\mathbf{D}$ will become a TTO that will operate on the TT of $\mathbf{u}$. But first, $\mathbf{u}$ must be written as a tensor of dimension $d \geq 2$. The choice of how to fold the vector $\mathbf{u}$ into a higher dimensional tensor depends on the problem; we discuss this for our application in Section 3.1.2 and Appendix A. For now, say that we have chosen to fold $\mathbf{u}$ into a 2-dimensional tensor $\mathcal{U} \in \mathbb{R}^{n \times m}$ (this particular choice of shape is arbitrary for the purposes of the example, but such a choice could be used to represent 2-dimensional data). Then, the appropriate shape for the TTO of $\mathbf{D}$ is $[n, n, m, m]$. This TTO can be computed using a process analogous to TT-SVD [1, Algorithm 1], or with Algorithm 1 given below; call the resulting TTO, $\mathcal{D}_{\text{TTO}}$. In line with the notation from Definition 5, $\mathcal{D}_{\text{TTO}}$ has two cores with shapes $[r_0, n, n, r_1]$ and $[r_1, m, m, r_2]$ respectively. The process of tensorizing $\mathbf{u}$ and $\mathbf{D}$ corresponds to (C) in Fig. 1, and computing their TT representations corresponds to (D). Together, all this allows us to write

$$\frac{\partial \mathcal{U}_{\text{TT}}}{\partial t} = -\mathcal{D}_{\text{TTO}} \mathcal{U}_{\text{TT}}. \tag{12}$$

The linear operation $\mathcal{D}_{\text{TTO}} \mathcal{U}_{\text{TT}}$ returns a tensor train and is analogous to the matrix-vector multiplication $\mathbf{Du}$. As noted in Definition 5, $\mathcal{D}_{\text{TTO}}$ acts on $\mathcal{U}_{\text{TT}}$ by contracting each of its cores with the corresponding cores of $\mathcal{U}_{\text{TT}}$. Using the first core of each as an example, a tensor of shape $[r_0, n, n, r_1]$ is contracted with a tensor of shape $[\tilde{r}_0, n, \tilde{r}_1]$, along their third and second axes respectively, to produce a resultant tensor of shape $[r_0 \tilde{r}_0, n, r_1 \tilde{r}_1]$.

All this means that we can solve Eq. (12) without leaving the TT format. The time derivative can be treated with any given time-integration method. However, as noted above (see Table 1), operations between TTs result in rank growth, so TT-rounding needs to be applied at appropriate intervals. As an example, a TT aware version of the Forward Euler time-integration scheme would be given by

$$\begin{aligned} \mathcal{U}^*_{\text{TT}} &= \mathcal{U}^n_{\text{TT}} - \Delta t \, \mathcal{D}_{\text{TTO}} \mathcal{U}^n_{\text{TT}} \\ \mathcal{U}^{n+1}_{\text{TT}} &= \text{round}\left(\mathcal{U}^*_{\text{TT}}, \varepsilon\right), \end{aligned} \tag{13}$$

where $\varepsilon > 0$ is some user chosen value for the allowable error.

Table 2
Meshes used in this work. The number following QS in each mesh refers to the resolution in degrees of latitude and longitude used by each mesh. To avoid a mesh singularity at the north and south poles, the meshes have boundaries at 80° north and 80° south. All domains have a radius of 6371220 m.

Mesh Name	Number of Cells	Number of Edges	Number of Vertices
QS1.6	22,500	45,225	22,725
QS0.8	90,000	180,450	90,450
QS0.4	360,000	720,900	360,900
QS0.2	1,440,000	2,881,800	1,441,800
QS0.1	5,760,000	11,523,600	5,763,600

With the example outlined above, we see that TT-methods are related, but not equivalent, to traditional reduced order methods (ROMs). Projection-based ROMs seek to discover a space of reduced dimension in which to evolve an approximation of the original model [24]. This is generally achieved via principle orthogonal decompositions (PODs) of a Galerkin (weak) formulation, or some other basis-reduction technique [25]. Other, data-driven methods such as the dynamic mode decomposition proposed by Schmid [26] provide a way to represent flow-data via modes from a reduced-dimensional space determined directly from data. Here, our SVD-based TT methods apply successive truncated SVDs directly to model-state data, approximating that data in a compressed format. The compressed model state can then be evolved in time, then uncompressed to recover (approximate) model-states from the original, full-dimensional, space. The fundamental difference here is that TT-solutions live in the original uncompressed space, whereas traditional ROM-solutions live in a space of reduced dimension. For our work here, the fundamental question is whether intermediate, TT-compressed, representations of relevant SWE model states have a low TT-rank structure that can be exploited for computational speedups.

3. Methods

3.1. An E3SM-like shallow water model

The particular shallow water model that we use here, swe-python [27], is written to use the same TRiSK spatial discretization [10] and meshes as the ocean component of the US Department of Energy's Energy Exascale Earth System Model (E3SM) [9]. This choice was made to evaluate TT methods in a setting as close to that of E3SM as possible. This model was originally written to run on unstructured, boundary-less, spherical Voronoi grids [17,18] consisting primarily of hexagons (Fig. A.1). However, the unstructured nature of these types of grids leads to significant difficulties applying tensor train to a tensor representing the state; we will discuss this further in Section 3.1.2 and Appendix A. In order to utilize the existing code, and remain in an E3SM-like setting, we have generalized swe-python to accept meshes consisting of quadrilateral cells and mesh boundaries as pictured in Fig. 4. At the boundary, we enforce a no-flow, no-slip boundary condition, which means that the normal velocity $\mathbf{u}$, as well as the tangential velocity $\mathbf{u}^{\perp}$ is zero. This gives us a structured grid on which we can apply our TT methods.

As the performance of TT methods can heavily depend on problem size, we will perform experiments as we vary mesh resolution, using a total of five meshes that we refer to as the prefix QS, meaning "Quadrilateral Sphere," followed by the resolution of the mesh in degrees latitude and longitude. Table 2 gives the resolutions and sizes of each of the meshes used. The resolutions are chosen so that moving from one mesh to the next in resolution increases the problem size by a factor of four.

3.1.1. Discrete spatial operators

An important aspect of the swe-python code is that all discrete spatial operations are implemented as sparse matrix-vector multiplications (SpMV) using SciPy [28], which has two primary benefits. First, SciPy uses a custom SpMV kernel written in C, which is significantly faster than Just In Time (JIT) compiled loops over spatial operator stencils, leading to improved computational performance comparable to that of optimized codes[1]. Second, as the TRiSK spatial operators are represented as matrices (2-dimensional tensors), they can be directly converted to TTOs of whatever appropriate shape.

As swe-python, and the ocean component of E3SM, implement only explicit time-stepping schemes, the implementation of spatial operations in the TT format is strightforward – the right-hand-side terms of a spatially discretized PDE can be evaluated by a combination of nonlinear element-wise operations and linear transformations. Example 1 provides a brief example of how this can be written. Note that while Example 1 uses explicit Euler time-stepping, the TT methods shown here are agnostic to the choice of time-stepping scheme; one may simply choose to apply TT-rounding at different points within a given scheme.

[1] By this, we mean that the SciPy SpMV kernel is expected to perform comparably to a similar kernel in a HPC code, not that swe-python's performance, as a single threaded Python application, is itself comparable to that of optimized, parallel code.

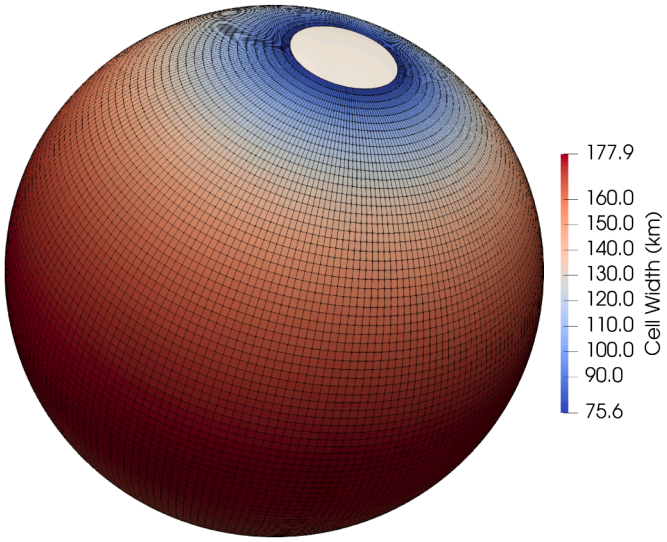


Fig. 4. A spherical TRiSK domain consisting of quadrilateral cells (QS1.6 from Table 2). Cells are colored according to the average width of each cell in km.

3.1.2. *Tensor train on structured and unstructured grids*

As alluded to above, the unstructured, variable resolution Voronoi grids normally used with TRiSK (Fig. A.1) are not conducive to favorable TT-compression. We will discuss this further in Appendix A, but will summarize the relevant issues here for completeness of the main text.

The tensor train decomposition of a tensor can be thought of as a separation of variables over the modes of the tensor. In the case of structured grids of any given dimension d, this is most commonly realized as a separation of variables over non-temporal dimensions. For example, a tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ might represent a model state in a 3-dimensional domain, with the first, second, and third tensor modes corresponding to spatial coordinates x, y, and z respectively. Using the notation of Definition 3, we could then approximate $\mathcal{X}$ in the TT-format as

$$\mathcal{X}_{\mathrm{TT}}(i_1, i_2, i_3) = \mathbf{G}_1(i_1)\, \mathbf{G}_2(i_2)\, \mathbf{G}_3(i_3)\,. \tag{14}$$

In this context, each core $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_3$ corresponds to the x, y, and z spatial dimensions respectively, and the size of rank $\mathbf{r} = [1, r_1, r_2, 1]$ encodes how effectively the model state can be represented in this separated state; low rank means that the state is approximately separable, while high rank means it is not, requiring more interaction between the cores to resolve the state.

Applying this strategy to our set of structured 2-dimensional meshes (e.g. Fig. 4) is straightforward. Let n_{lat} be the number of grid points in the latitudinal direction and let n_{lon} be the number of grid cells in the longitudinal direction. The thickness variable h is realized as a 2-dimensional tensor $\mathcal{H} \in \mathbb{R}^{n_{\mathrm{lat}} \times n_{\mathrm{lon}}}$, where the data is arranged so that the first mode corresponds to each cell's latitude and the second mode corresponds to each cell's longitude. The velocity variable $\mathbf{u} = (u, v, 0)$ is realized as a 2-dimensional tensor $\mathcal{U} \in \mathbb{R}^{(2n_{\mathrm{lat}}+1) \times n_{\mathrm{lon}}}$, where u and v are the fluid velocities in the longitudinal and latitudinal directions respectively. The data for $\mathbf{u}$ is arranged similarly to that for h so that the first mode corresponds to each edges' latitude and the second mode corresponds to each edges' longitude; this has the effect that the rows of $\mathcal{U}$ alternate between values for v and values for u. Then, when $\mathcal{H}$ and $\mathcal{U}$ are approximated by TTs, $\mathcal{H}_{\mathrm{TT}}$ and $\mathcal{U}_{\mathrm{TT}}$. These can be viewed as separating variables over latitude and longitude similar to Eq. (14).

The problem with applying TT methods to an unstructured Voronoi mesh as in Fig. A.1 is that the data for both h and $\mathbf{u}$ must be represented by a tensor in a way that some separation of variables can be taken advantage of. The unstructured, variable resolution grids used in E3SM do not have an obvious mapping to a tensor space that preserves the structure of the data in a way that allows for an advantageous splitting of variables. There are several ways that this problem could be dealt with: Cleverly sorting and mapping the grid points, re-gridding the data, or using a different tensor decomposition method that is possibly more amenable to an unstructured mesh. However, each of these strategies introduce their own set of challenges; we discuss further in Appendix A.

3.2. *Working with sparse tensor train operators*

As described in Section 3.1.1, the TRiSK spatial operators are treated as sparse matrices which are then converted to TTOs. It is possible to achieve this via standard algorithms such as those presented in Oseledets [1], but most methods and implementations assume that the input tensor is dense. This is fine for small problem sizes, but as the problem size grows, our TRiSK operators can only be reasonably stored and manipulated in a sparse representation.

To facilitate the rest of our work, we have implemented a collection of TT algorithms from Li et al. [29] meant for working with sparse input tensors. Our Python implementation uses PyData Sparse [30], which generalizes SciPy sparse arrays to arbitrary

dimensions. Our code is available on GitLab [31]. We will briefly describe the requisite algorithms from Li et al. [29] before describing our approach to obtain a TTO from a sparse matrix operator in Algorithm 1.

Algorithm Li-4: Compute an exact TT that has sparse cores [29, Algorithm 4].

Input: A sparse tensor $\mathcal{A} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ and an integer p with $1 \leq p \leq d$.

Output: A TT-format tensor $\mathcal{A}_{\mathrm{TT}}$ with sparse cores that is equivalent to $\mathcal{A}$, possibly with large TT-rank. All cores of $\mathcal{A}_{\mathrm{TT}}$, except for the p-th core, consist of only ones and zeros.

Algorithm Li-6: Efficient TT-rounding [29, Algorithm 6].

Input: A TT, $\mathcal{A}_{\mathrm{TT}} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$, the desired error ε, and an integer p with $1 \leq p \leq d$. In the case that $\mathcal{A}_{\mathrm{TT}}$ was computed using Algorithm Li-4, p should be chosen to be the same as in that process for the best performance.

Output: A TT, $\mathcal{B}_{\mathrm{TT}}$ approximating $\mathcal{A}_{\mathrm{TT}}$ such that $\|\mathcal{A}_{\mathrm{TT}} - \mathcal{B}_{\mathrm{TT}}\|_{\mathrm{F}} < \varepsilon \|\mathcal{A}_{\mathrm{TT}}\|_{\mathrm{F}}$.

Algorithm 1: Compute a TTO representation of a sparse matrix operator.

Input: A sparse matrix operator $\mathbf{A} \in \mathbb{R}^{m \times n}$ for $m, n \in \mathbb{N}$, the desired accuracy $\varepsilon > 0$, and $m_1, n_1, \cdots, m_d, n_d \in \mathbb{N}$ giving the desired shape of the TTO $[m_1, n_1, \cdots, m_d, n_d]$ that will act as a map $\mathbb{R}^{n_1 \times \cdots \times n_d} \to \mathbb{R}^{m_1 \times \cdots \times m_d}$.

Output: $\mathcal{A}_{\mathrm{TTO}} \in \mathbb{R}^{m_1 \times n_1 \times \cdots \times m_d \times n_d}$ is a TTO approximating the action of $\mathbf{A}$ to accuracy ε.

Reshape $\mathbf{A}$ into a tensor $\mathcal{A}$ with shape $[m_1, m_2, \cdots, m_d, n_1, n_2, \cdots, n_d]$.
Permute the modes of $\mathcal{A}$ such that its new shape is $[m_1, n_1, m_2, n_2, \cdots, m_d, n_d]$.
Reshape $\mathcal{A}$ to have shape $[m_1 n_1, m_2 n_2, \cdots, m_d n_d]$.
Apply Algorithm Li-4 to $\mathcal{A}$ with $p = \lfloor d/2 \rfloor$ to obtain a TT, $\mathcal{A}_{\mathrm{TT}}$ with sparse cores.
Apply Algorithm Li-6 to $\mathcal{A}_{\mathrm{TT}}$ with $p = \lfloor d/2 \rfloor$ and desired accuracy ε to obtain an approximate TT, $\mathcal{B}_{\mathrm{TT}}$ with cores $\mathcal{G}_k$ for $k = 1, \cdots, d$, each of which has shape $[r_{k-1}, m_k n_k, r_k]$.
for $k = 1, \cdots, d$ **do**
    Permute the modes of $\mathcal{G}_k$ such that its new shape is $[m_k n_k, r_{k-1}, r_k]$.
    Reshape $\mathcal{G}_k$ to have shape $[m_k, n_k, r_{k-1}, r_k]$.
    Permute the modes of $\mathcal{G}_k$ such that its new shape is $[r_{k-1}, m_k, n_k, r_k]$.
return *The cores $\mathcal{G}_k$ for $k = 1, \cdots, d$, which form a TTO $\mathcal{A}_{TTO} \in \mathbb{R}^{m_1 \times n_1 \times \cdots \times m_d \times n_d}$.*

Example 2. Using Algorithm 1, we can convert an arbitrary sparse matrix operator, and therefore any linear operator, to the TTO format. To give a concrete example that is used heavily in Section 4, consider an arbitrary QS grid with n_{lat} grid points in the latitudinal direction, and n_{lon} grid points in the longitudinal direction. Let $\mathbf{D}$ be the TRiSK divergence operator, which maps the fluid velocity $\mathbf{u}$ on grid edges to cell-centered values for the divergence. The vector $\mathbf{u}$ has length $(2n_{\mathrm{lat}} + 1)n_{\mathrm{lon}}$ and $\mathbf{D}$ is a sparse matrix of shape $[n_{\mathrm{lat}} n_{\mathrm{lon}}, (2n_{\mathrm{lat}} + 1)n_{\mathrm{lon}}]$. As described in Section 3.1.2, the fluid velocity can then be represented as a TT, $\mathcal{U}_{\mathrm{TT}} \in \mathbb{R}^{(2n_{\mathrm{lat}}+1) \times n_{\mathrm{lon}}}$. We apply Algorithm 1 to the matrix $\mathbf{D}$ to obtain a TTO, $\mathcal{D}_{\mathrm{TTO}} \in \mathbb{R}^{n_{\mathrm{lat}} \times (2n_{\mathrm{lat}}+1) \times n_{\mathrm{lon}} \times n_{\mathrm{lon}}}$. Then, we can write

$$\mathcal{D}_{\mathrm{TTO}} \mathcal{U}_{\mathrm{TT}} = \mathcal{V}_{\mathrm{TT}}, \tag{15}$$

where the result $\mathcal{V}_{\mathrm{TT}} \in \mathbb{R}^{n_{\mathrm{lat}} \times n_{\mathrm{lon}}}$ is a TT giving the cell-centered values of the divergence of the fluid velocity. We can write an expression similar to that in Eq. (15) for all TRiSK spatial operators.

4. Experiments

4.1. Experiment design

The purpose of the experiments that follow are to evaluate the performance of TT-format operations in context of geophysical fluid dynamics (GFD). We are primarily concerned with two metrics. First is the TT-compression (Definition 4) and how this relates to the complexity of the model state in time. Second is the speedup achieved by the TT-format operations versus standard (non-TT) methods – specifically as compared to an optimized sparse matrix-vector multiplication kernel.

4.1.1. Timing linear operators

For practical reasons that will become clear in Section 4.3, we have opted to perform our TT performance experiments in an "offline" fashion. By this, we mean that we have opted not to implement a fully TT-enabled version of swe-python that solves the SWEs in the TT-format. Instead, we solve the SWEs using standard methods while writing the model state to disk at intervals. Then, we apply our TT-format operations to the model state at each saved time-level. This allows us to quickly test different TT configurations

without the need to run the model from scratch each time. It has been shown in Danis et al. [8] that TT-methods can successfully maintain the formal order of accuracy of the methods on which they are based, in both space and time. *Therefore, our concern is not whether these types of methods can be used to accurately solve the SWEs, but rather if they can achieve a significant speedup over standard methods and therefore be used in real-world application HPC codes.*

In swe-python, each time-step requires a series of linear transformations on the model state and a small number of element-wise nonlinear operations. As such, the majority of the computational cost comes from operations of the form of large matrix-vector multiplications. It is therefore sufficient to test the performance of our TT methods for these types of operations, which is then representative of the model's overall computational performance. Of course, in a realistic code, there is additional nontrivial computational overhead primarily in the form of I/O and parallel communication (e.g. MPI). While we have not conducted extensive testing with TT methods in such a production-environment, we expect that the these overhead costs for TT methods would not exceeded those incurred by standard methods; since a TT represents a compressed model state, communication and related operations should cost less, being proportional to the TT compression rate.

In Section 4.3, we present timing and error results of taking the divergence of the model velocity field through various methods; sparse matrix-vector multiplication (SpMV), a simple Just In Time compiled Python loop over the operator stencil (JITLoop), and the TT-format linear transformation described in Example 2 (TTMV). Our choice to use the divergence operator as a representative operation is essentially arbitrary since, again, all linear operations are given in the form of matrix-vector multiplications. However, we were motivated to choose the divergence operator as it is a map between edge-centered and cell-centered quantities, and so interacts with both grid locations that hold prognostic variable quantities. The SpMV case is representative of an optimized, high performance implementation, while the JITLoop case is representative of a more naïve implementation – the comparison to both helps to more clearly contextualize the performance of our TT-methods.

As discussed previously, the SpMV kernel is supplied by SciPy [28]. Additionally, we use Numba [32] for the JIT compilation of JITLoop and torchTT [33] for TT operations, which in turn uses PyTorch [34] as a backend. All experiments were carried out on a Linux machine with two Intel Xeon Silver 4215R CPUs, two Nvidia Quadro RTX 8000 GPUs, and 64 GB of memory. This machine was provided by the Center for Nonlinear Studies (CNLS) at Los Alamos National Laboratory (LANL), and has been affectionately dubbed the Computer for Nonlinear Studies. For consistency across the various Python libraries used in this work, no attempt at parallelization was made within swe-python or the deployed tensor train libraries; Python was run as default in serial. Also, we note that results reported below using this machine's GPU were obtained by running on a single GPU. GPU operations were performed with PyTorch as backend, and timed as shown in Listing 1. Finally, we note that when timing all methods, SpMV, JITLoop, and TTMV, the reported timings are the average of 10 total runs, with one additional "warm-up" run at the beginning which is not counted in the average.

```
# send data to GPU in advance of operations
tto_gpu = tto.to('cuda')
state_tt_gpu = state_tt.to('cuda')

# create cuda event trackers
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)

# perform and time TT matrix-vector multiplication
start.record()
result_tt = tto_gpu @ state_tt_gpu
end.record()
torch.cuda.synchronize()

# record time, converting milliseconds to seconds
matvec_time = start.elapsed_time(end) / 1000

# perform and time TT-rounding
start.record()
result_tt = result_tt.round(eps=dynamic_eps)
end.record()
torch.cuda.synchronize()

# record time, converting milliseconds to seconds
round_time = start.elapsed_time(end) / 1000
```

Listing 1: GPU timing for TT operations using torchTT and PyTorch.

4.1.2. Choosing ϵ

The computational efficiency of the TTMV as compared to that of the SpMV depends on the rank of the state TT being operated on, which in turn depends on the chosen TT approximation error ϵ (e.g. see Eq. (4)). In our experiments, we choose ϵ in a way that would guarantee the preservation of formal order of accuracy (if the TT-methods were being applied in the full model), as described in Danis et al. [8, Section 3d];

$$\epsilon = \min\left(10^{-3},\, C_\epsilon \frac{V^{1/2}\Delta x^{\ell-1/2}}{\max_{\mathbf{q}\in\{\mathbf{h},\mathbf{u}\}} \|\mathbf{q}\|_F}\right), \tag{16}$$

where V is the volume of the domain, ℓ is the spatial order of accuracy of the spatial discretization (2 in our case), $\mathbf{h}$ and $\mathbf{u}$ are vectors of the thickness and velocity model state, and C_ϵ is a problem dependent variable discussed below.

The presence of C_ϵ, which must be chosen by the user empirically, creates some difficulty in determining a suitable value for ϵ. Ideally, we would choose C_ϵ so that the error induced by the TT-approximation of the state was less than the temporal and spatial truncation errors, but these cannot be known *a priori*, so it typically needs to be hand-tuned. Large C_ϵ will allow the TT-approximation to achieve low rank and therefore high computational efficiency, but may not represent the model state with sufficient accuracy. Small C_ϵ can assure high accuracy, but can lead to very high TT-rank. Danis et al. [8] suggests choosing C_ϵ via a process analogous to a grid-independence study, where C_ϵ is taken lower and lower until it no longer has an effect on the model solution accuracy. As we are not solving the full SWEs in the TT-format, we instead choose a fixed $C_\epsilon = 1$ for all test cases, which was observed by Danis et al. [8] to be sufficient in the majority of cases. We will share results for other choices of C_ϵ in Appendix B for completeness.

The central goal of this work is to determine whether TT-methods can supply speedups in cases close to realistic configurations. The work from Danis et al. [8] has shown that TT-methods can achieve arbitrary accuracy; in particular, they showed that TT-methods could recover formal order of accuracy when compared to a test case with an analytical solution. Here, we select the error ϵ so as to ensure accuracy, but seek to answer the question of whether the solutions to these problems have a low TT-rank structure that can be exploited for speedups.

4.2. Test cases

To investigate the performance of TT-format operations across a range of problems relevant to GFD, we consider a suite of four test cases of increasing complexity. All test cases take place on a rotating sphere with angular velocity $\Omega = 7.292 \times 10^{-5}$ s^{-1} and a radius of 6371.22 km. The sea-floor topography is uniform unless otherwise specified. Snapshots of the final states for each test case are given in Fig. 21.

Quasi-Linear Gravity Wave (QLW): A simple gravity wave traveling from 0° latitude, to the north/south boundaries, then back again, a process which takes approximately seven days of simulated time. This is the case of an external wave mode, also called the Lamb mode in the context of atmospheric modeling. The momentum advection terms in the momentum component of Eq. (1) are turned off, making the momentum equation purely linear. The thickness equation component of Eq. (1) is formally nonlinear, but because the fluid depth is sufficiently large compared to any perturbation in the thickness, this behaves as though it is linear. Therefore, we refer to this test case as quasi-linear; using the same notation as Eq. (1), the model equations are given by

$$\begin{aligned} \frac{\partial \mathbf{u}}{\partial t} &= -g\nabla(h + z_b) \\ \frac{\partial h}{\partial t} &+ \nabla\cdot(h\mathbf{u}) = 0\,. \end{aligned} \tag{17}$$

The fluid velocity is initialized to zero, while the thickness is initialized as a Gaussian bell curve centered at 0° latitude of the form $e^{-100x^2-100y^2} + 500$. The case runs for seven simulated days.

Geophysical Gravity Wave (GGW): The same initial conditions as QLW, but with the full nonlinear SWEs. The flow exhibits similar behavior, but the final state, influenced by the nonlinear momentum advection and Coriolis terms, is more complex. The case runs for seven simulated days.

Barotropically Unstable Jet (BUJ): Originally formulated by Galewsky et al. [12], this case consists of a strong zonal flow at mid-latitude with a small perturbation that causes the barotropic jet to become unstable. This flow is driven by strongly nonlinear momentum advection tendencies, physical processes that are of particular relevance to weather prediction and climate modeling. The case runs for six simulated days.

Williamson Test Case 5 (WTC5): A zonal flow over an isolated mountain, one of the standard Williamson et al. [13] SWE test cases. Here, the flow is initialized to a steady state assuming uniform bottom topography, but the bottom topography is then altered to introduce an isolated mountain at mid-latitude. This perturbation alters the geostrophic balance of the flow, triggering the development of advection-driven instabilities. The case runs for 50 simulated days.

This set of test cases was chosen to showcase a wide range of tensor train performance scenarios. Danis et al. [8] has already shown that TT-methods can achieve speedups while maintaining formal order of accuracy. Here, our goal is somewhat orthogonal to that of Danis et al. [8]; we aim to show cases where TT-methods work, and cases where they do not. The QLW case showcases TT-performance in the simplest possible case on the sphere, to provide a baseline for the tests that follow. The GGW case represents the simplest case that can be used in a real-world application (e.g. tsunami modeling and coastal inundation). Additionally, since GGW differs from QLW only in that it uses the full nonlinear SWEs, this shows that nonlinearity alone does not greatly effect TT-performance. The BUJ case is perhaps the most important of the four cases, as it begins with a simple state which is shown to achieve

strong TT-compression, then evolves into a more complicated state where TT struggles to find a low rank representation. Finally, we end with WTC5 as one of the most used SWE test cases in the literature, to show TT-performance on a realistic problem that has been widely studied. Taken together, the results from these four test cases provide a clear narrative about the effectiveness of TT-methods for this application.

4.3. Results

Here we present the results of TT-method performance experiments on the four test cases of increasing complexity described in Section 4.2. As discussed previously, we present the results of two primary metrics; first, the TT-compression of both the layer thickness and velocity variables in time, assuming a dynamically chosen ϵ as in Eq. (16), and second, the speedup achieved by TT-methods versus an optimized SpMV kernel of performing a representative linear transformation on the model state data (in this case, taking the divergence of the velocity field). As TT-rounding is an important contributor to the overall performance of a TT-method, a rounding step is accounted for in the timing results. We additionally give some results not accounting for the rounding step which are clearly marked.

When comparing a TT-obtained solution to the reference solution given by standard methods, we use a normalized Frobenius error, defined as

$$E_{\mathrm{NF}} = \frac{\|\mathbf{s} - \mathbf{t}\|_{\mathrm{F}}}{\|\mathbf{s}\|_{\mathrm{F}}}, \tag{18}$$

where $\mathbf{s}$ is the reference solution, $\mathbf{t}$ is the TT-method solution.

Finally, to connect the TT-compression to the complexity of the fluid flow through a metric relevant to geophysical fluids, we also compute the total eddy kinetic energy (EKE) in time on QS0.2. EKE can be formally defined in different ways, but is generally understood to be the kinetic energy of a perturbation to the mean fluid flow. Here, we use

$$K_{\mathrm{EKE}} = \frac{1}{2}\left((\bar{u} - u)^2 + (\bar{v} - v)^2\right), \tag{19}$$

where $\mathbf{u} = (u, v, 0)$ is the fluid velocity and $\bar{\cdot}$ denotes an average in time equal to the length of each test case. We tested different formulations for EKE, but got qualitatively similar answered for the cases considered here. Then, as we plot the total EKE, we simply integrate K_{EKE} over the spherical spatial domain S and normalize by the volume of S,

$$T_{\mathrm{EKE}} = \frac{1}{\mathrm{vol}(S)} \int_S K_{\mathrm{EKE}} \mathrm{dA}\,. \tag{20}$$

We only plot the EKE on the QS0.2 because the EKE is qualitatively similar on each mesh, and this is the highest resolution mesh by all test cases. The only difference between resolutions is the order of magnitude of the EKE, which increases with resolution in this range.

The purpose of our calculation of the total EKE is to provide a diagnostic quantity common to GFD applications that we can compare to the TT-rank over the course of each simulation. Below, we observe that this is a potentially useful metric that may suggest whether certain flows can be well compressed with TT; in our results, high total EKE corresponds to high TT-rank. However, we note that it is not true in general that high EKE implies high TT-rank, or vice-versa. One can easily contrive examples where a given state exhibits high EKE and low TT-rank, as well as examples with low EKE and high TT-rank. For example, the initial state of the BUJ case has low TT-rank; if we were to run this case without the thickness perturbation so the barotropic jet was stable for all time, but we suddenly forced the jet to the south during the final hours of the simulation, the resulting flow would have the same low TT-rank but high EKE. As such, we include EKE results and plots not to make a general claim connecting high EKE and high TT-rank, but instead to suggest that it may provide a valuable diagnostic for a certain class of flows.

4.3.1. Quasi-linear wave

The first and simplest of our four test cases. The momentum advection and Coriolis terms are turned off, leaving us with a quasi-linear system; the model state in QLW is entirely characterized by a small number of superimposed gravity waves. The case starts with a single gravity wave, but more are generated when the original collides with the domain boundaries at the north and south poles, resulting in a partial reflection back towards the original starting point.

Fig. 5 shows that both the TT of the layer thickness and normal velocity data remain well compressed for the entirety of simulation. Further, as expected, the compression rate improves as the mesh resolution increases. Of course, the overall rank of the TT approximations to these data both increase with resolution, but this is simply explained by the overall problem size increasing by a factor of four between each of our meshes. As can be seen in both the compression curves and rank curves, there is a clear structure where the curves rise and dip over the course of the QLW simulation. The four troughs occur when the gravity wave first contacts the north/south pole border, when the unreflected portion of the wave coalesces with itself on the opposite side of the globe, when it contacts the border again, and finally when the wave returns to its starting position.

Again as expected, the TT-error (as defined in Eq. (18)) improves with mesh resolution as the dynamic ϵ is restricted to account for theoretically smaller spatial truncation errors. We note that for the normal velocity data on QS1.6 and QS0.8, the dynamically chosen ϵ defaults to around 10^{-3} as defined in Eq. (16). This is explained by the fact that the grid resolution in both cases is simply large enough that the maximum allowed ϵ is used. Compare this to Fig. B.1 where we take $C_\epsilon = 0.1$ and see that the ϵ chosen for the two meshes is in fact different.

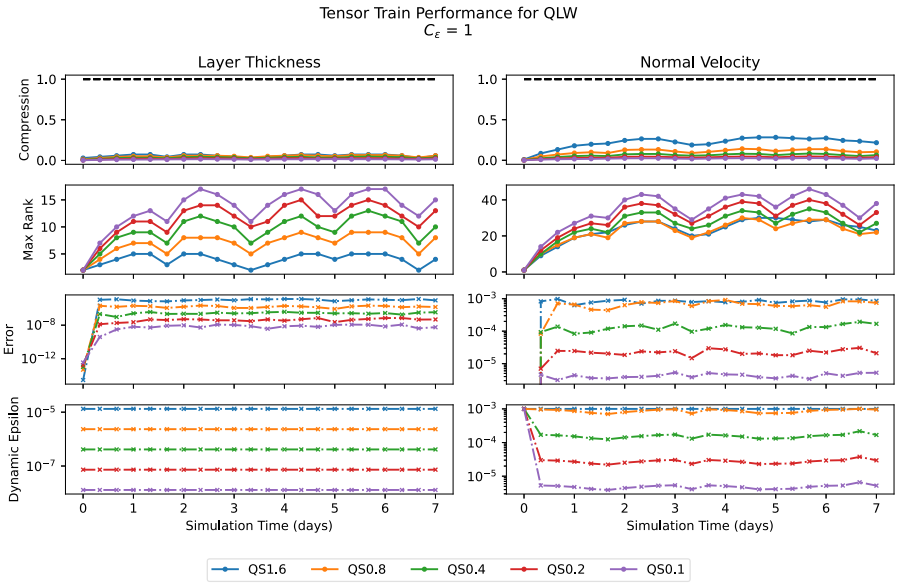


Fig. 5. TT-compression results for QLW with $C_\epsilon = 1$. Note that the black dashed lines in the compression plots on the top row are there to show a compression rate of 1, after which the TT costs more to store than original tensor. This marker also appears in all similar plots that follow.

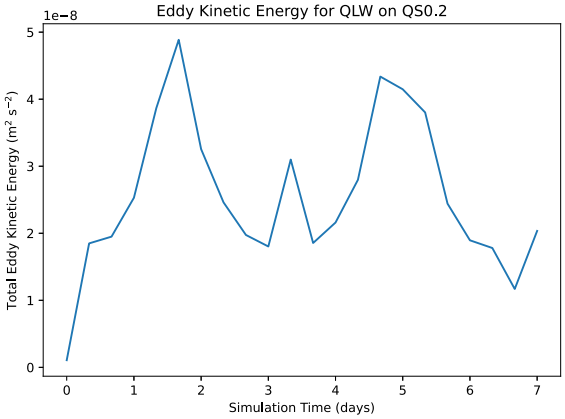


Fig. 6. Total eddy kinetic energy for QLW on QS0.2.

Fig. 6 shows the total EKE over the course of the QLW case. We see that the energy of the perturbation on the mean flow remains very small for the entirety of the simulation (note that the scale of the vertical axis is 10^{-7}). In particular, compare this to similar plots for BUJ and WTC5 in Figs. 14 and 18 respectively.

Next, Fig. 7 shows the speedup for our TT-methods at three different points in the simulation; at the beginning, approximately half-way through, and at the end. This is done to show the speedup achieved at different levels of TT-compression/rank. An overview of the speedup over the course of the entire simulation on QS0.1 is given in Fig. 8. QS0.1 was chosen as a representative mesh because it is by far the largest problem, where the TT-methods perform best.

Throughout the simulation, TTMV outperforms SpMV by a factor of approximately three, except at the very beginning of the simulation, where the fluid velocity is initialized to zero so the TT-approximation to the velocity data is trivial (Fig. 7a and b). Both TTMV and SpMV outperform JITLoop, which serves as a worst-case point of comparison. The speedup obtained by TTMV as compared to JITLoop is approximately $3/0.18 = 16.\overline{66}$. This serves to demonstrate that the implementation to which we compare TT-methods is of the utmost importance when considering possible speedups.

4.3.2. Geophysical gravity wave

This second test case is initialized the same as QLW, except uses the full nonlinear SWEs. This serves to show that simply adding nonlinear dynamics to the model does not necessarily cause issues with TT-performance. The case behaves similarly to QLW, but

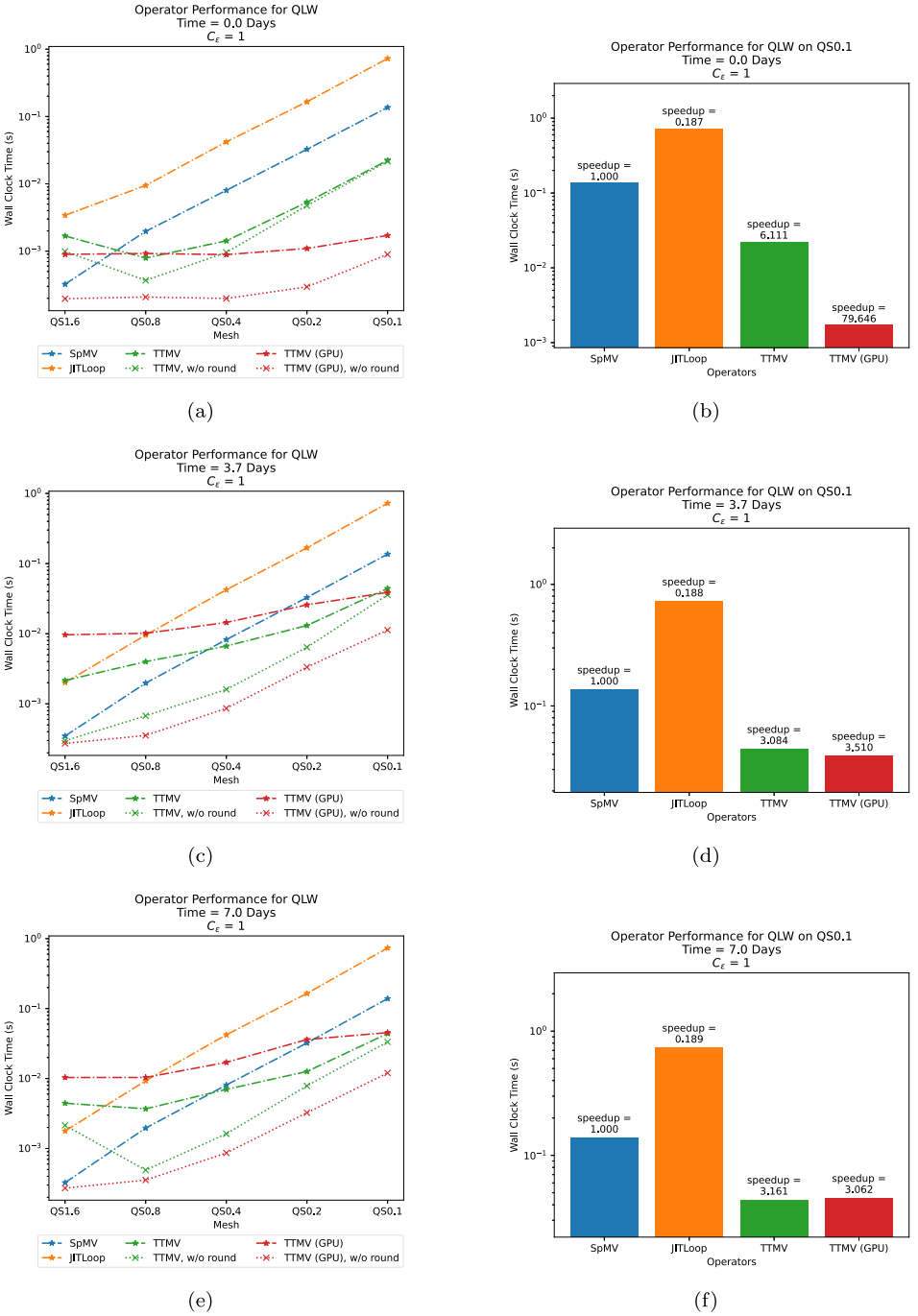


Fig. 7. TT-format operation performance in the QLW test case. Note that here, and in all similar performance plots that follow, all TT operations use the same value of C_ϵ and are therefore achieving the same accuracy. The SpMV and JITLoop operators are, of course, exact up to numerical truncation error.

the gravity waves spawned from the initial Gaussian perturbation to the layer thickness and the subsequent collision with the mesh boundaries are subject to nonlinear effects due to momentum advection and the rotation of the sphere.

As in Fig. 5 for QLW, Fig. 9 shows the TT-compression for layer thickness and normal velocity during the GGW test case. Similar to the previous case, both prognostic quantities remain well compressed throughout the simulation on sufficiently large problems. The compression is slightly less than in QLW however, as can be seen from the max rank plots in Fig. 9 as compared to those in Fig. 5;

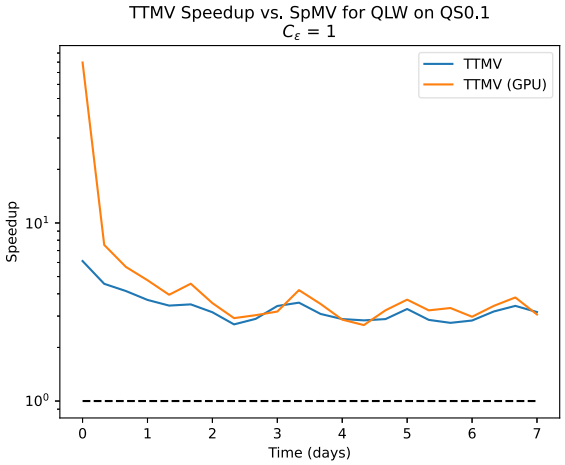


Fig. 8. TTMV speedup versus SpMV in time for the QLW test case on QS0.1.

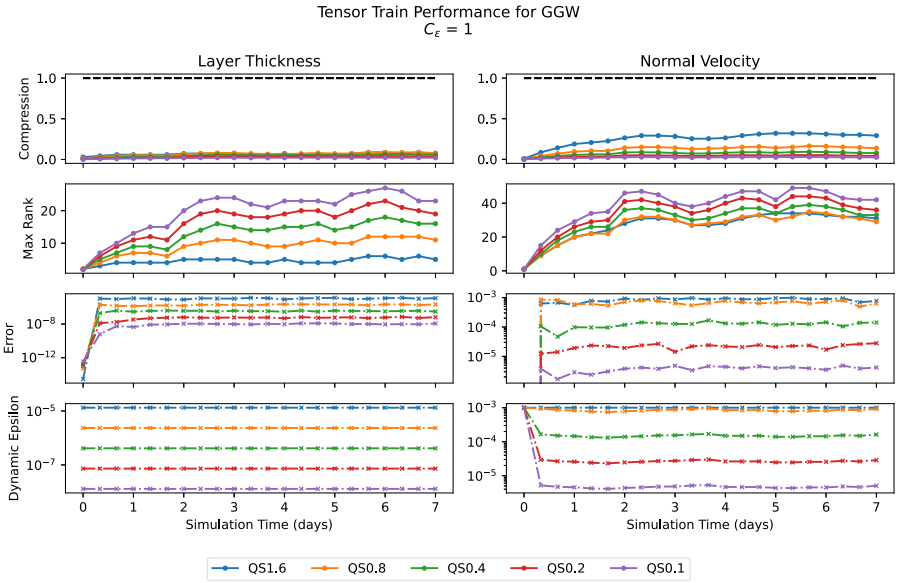


Fig. 9. TT-compression results for GGW with $C_\epsilon = 1$.

we can see that on QS0.1, the TT-ranks for both thickness and velocity are slightly higher, clearly due to the nonlinear dynamics. However, because these nonlinear dynamics are not dominant, the effect on the compression is relatively small.

As evidence that the nonlinear terms are not dominant in GGW, we can compare the total EKE plots between the present case (Fig. 10) and QLW (Fig. 6). Both show that the total EKE is on the order of 10^{-7} and show a similar evolution in time. Again, this shows that the presence of nonlinear dynamics in the problem does not necessarily lead to a loss of TT-compression.

Fig. 11 shows the speedup for our TT-methods at the same points in time in Fig. 7 for QLW. The results here are very similar to those from before, due to the fact that the TT-compression rates in both cases are also similar. For the same reason, Figs. 8 and 12 are qualitatively similar.

4.3.3. Barotropically unstable jet

Our third test case is a canonical case for the SWEs that begins with a very simple initial condition and evolves into a flow with structures at multiple scales relevant to realistic applications. As such, this is perhaps the most illustrative test case for our purposes. We expect that TT to effectively compress the initial states for both thickness and velocity, then will see how the TT-compression evolves in time with the flow.

As we can see in Fig. 13, the model state does indeed begin as well compressed, and remains so through approximately the first half of the simulation where the barotropic jet begins to roll up due to the perturbation in the initial condition. After the half-way mark however, the compression, especially for the velocity data, begins to worsen. By the end of the simulation, compression is lost

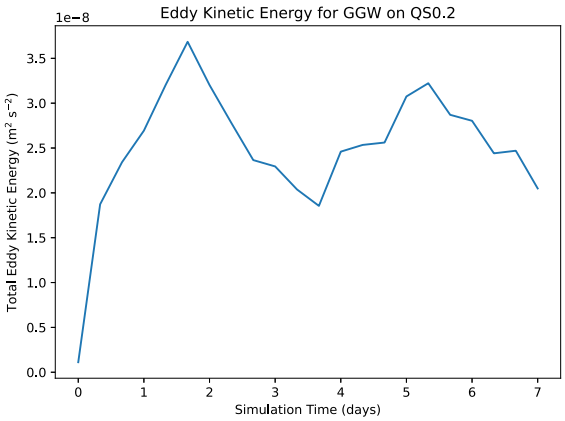


Fig. 10. Total eddy kinetic energy for GGW on QS0.2.

entirely. The TT-compression for the velocity even surpasses a ratio of 1, meaning that the TT of the data is larger than the original uncompressed data, exhibiting anti-compression. Notably, during the final two days of the simulation, the compression on the larger meshes (in particular QS0.1) is worse than the smaller meshes. This is likely because the high resolution meshes resolve smaller scale structures in the flow, requiring the TT-rank to grow more quickly to maintain fidelity – this can be seen in the plots of the maximum rank for both the thickness and velocity (Fig. 13).

It is also notable that the total EKE (Fig. 14) and the compression are positively correlated. The total EKE starts low as the flow remains close to its simple initial condition, but begins to quickly grow between days three and four as the jet begins its roll-up. This coincides with the loss of compression for both prognostic quantities.

The speedup achieved by TT in the BUJ case suffers similarly; we see nontrivial speedup during the first few days of the simulation, but the loss of compression results in TTMV being slower than SpMV for the majority of the run (Fig. 16), with the slowdown being particularly problematic at the end of the simulation. TTMV remains faster than JITLoop for the majority of BUJ (Fig. 15), but falls well behind near the end. Regardless, the comparison to the optimized SpMV gives a much more realistic approximation to a high performance code.

4.3.4. *Williamson test Case 5*

Our final test case exhibits the most complicated dynamics, and is run for a full 50 simulated days. Note that we have omitted results for WTC5 on QS0.1 because of the computational expense of running swe-python on this high resolution mesh.

Looking at Fig. 17, we can see that compression is lost very soon after the beginning of the simulation. As a reminder, the loss of compression is not instantaneous (i.e. after a single time-step), because we are only plotting the compression (and related quantities) at a subset of time-levels. Nonetheless, the loss of compression is very rapid, and does turn into anti-compression, especially for the TT of the normal velocity.

Compare Figs. 14 and 18, which show the total EKE for BUJ and WTC5 respectively. The EKE in the WTC5 case starts near 0.1, while the EKE in the BUJ is close to zero for much of the simulation, only reaching 0.1 around four and a half days. This four and a half day mark is also when TT-compression in the BUJ case begins to quickly degrade. That is, the total EKE for WTC5 starts at a level not reached by BUJ until it begins to lose TT-compression. Taken together with the results from the QLW and GGW test cases (which have EKE near zero), this points to a possible connection between EKE (i.e. the difference from the mean velocity state) and TT-compression in certain flows.

Because of the immediate loss of compression, the speedups achieved by TTMV are poor as compared to SpMV. Of course, we should note that the mesh on which Fig. 20 is generated is four times smaller than corresponding plots for the other three test cases; the shown speedup is likely worse than it would be on QS0.1. Nonetheless, the qualitative behavior of the speedup curves would likely be consistent across cases.

4.4. *Discussion*

Taking the results of all four test cases together, we see a clear progression in the effectiveness of our TT-methods. In the QLW case where the model dynamics are relatively simple for the entirety of the simulation, we see strong TT-compression and speedup compared to the optimized SpMV kernel. Adding nonlinear dynamics to the system in the GGW case, we see a slight increase in the TT-rank for both the layer thickness and normal velocity data, but the overall compression remains strong, leading to good speedup similar to that in the QLW case. The BUJ test case starts with a relatively simple state that shows itself to be “compressible” (in the sense of TT-compression), but develops to exhibit complex structures at multiple scales which require high TT-rank to accurately represent, resulting in poor compression and eventually anti-compression. Finally, while WTC5’s initial condition is compressible,

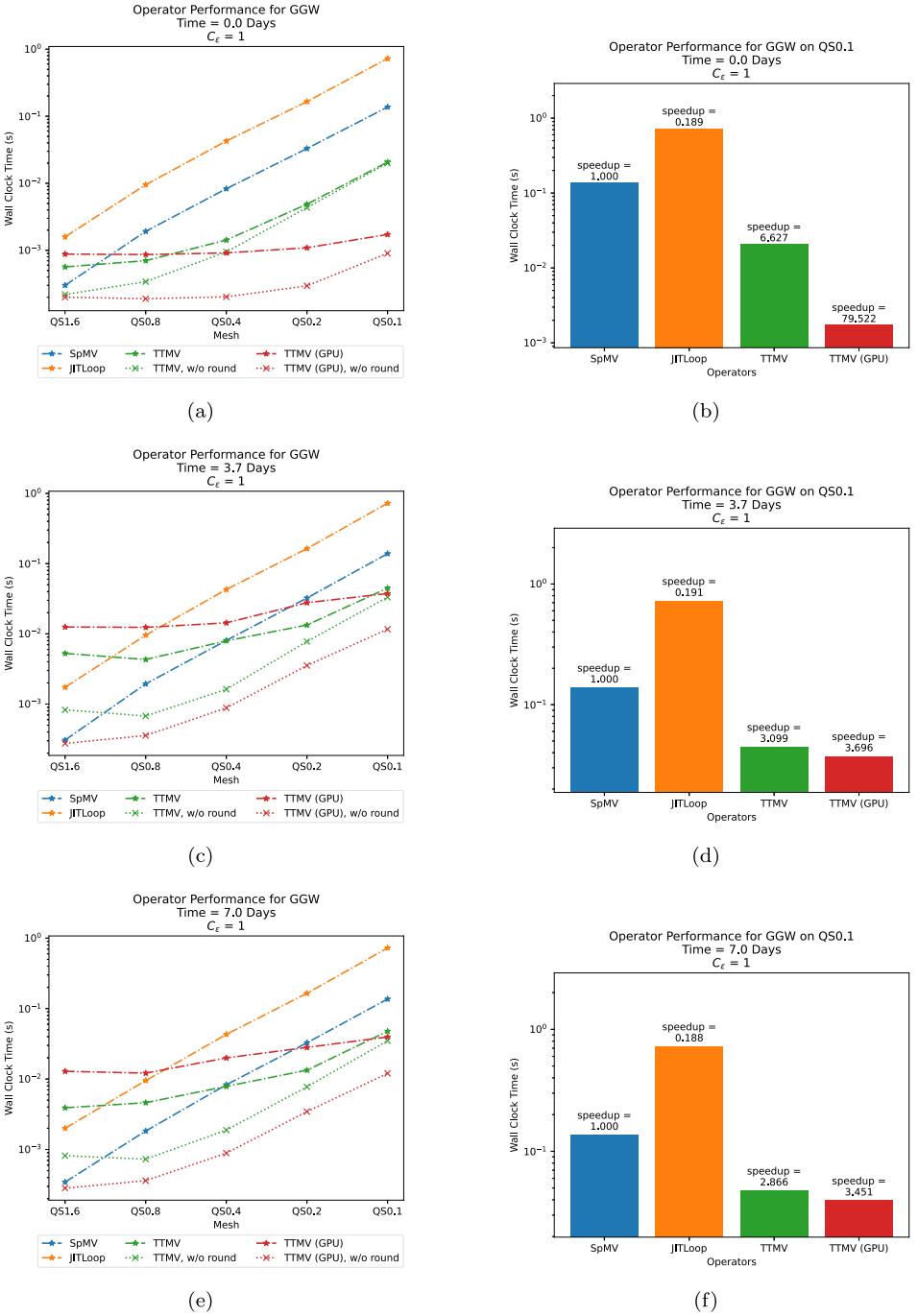


Fig. 11. TT-format operation performance in the GGW test case.

its state quickly develops so as to require very high TT-rank, leading to strong anti-compression. Fig. 21 serves as a summary and visualization of our main results.

Of course, the speedup obtained by TT-methods versus traditional methods is directly tied to the TT-compression of the state being evolved; as demonstrated above, the TT-compression is effectively representative of the obtainable speedup. An important takeaway from this fact is that the specific speedup results shown here need not be definitive for the overall results to hold. The actual speedups obtained depend on a number of highly variable factors including algorithm implementation, programming language, compilers, and computing hardware. For example, it is likely that our the reported TT speedups would improve over time as implementations

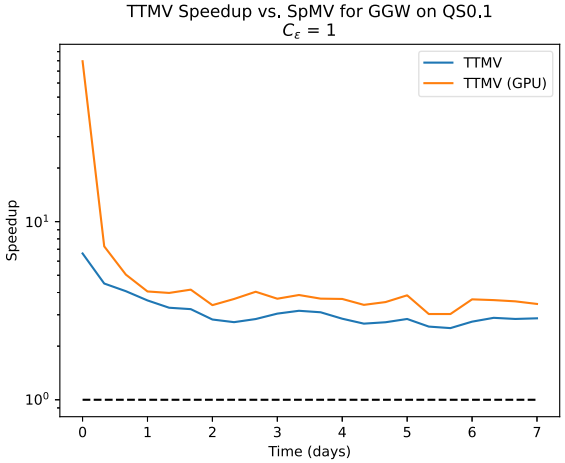


Fig. 12. TTMV speedup versus SpMV in time for the GGW test case on QS0.1.

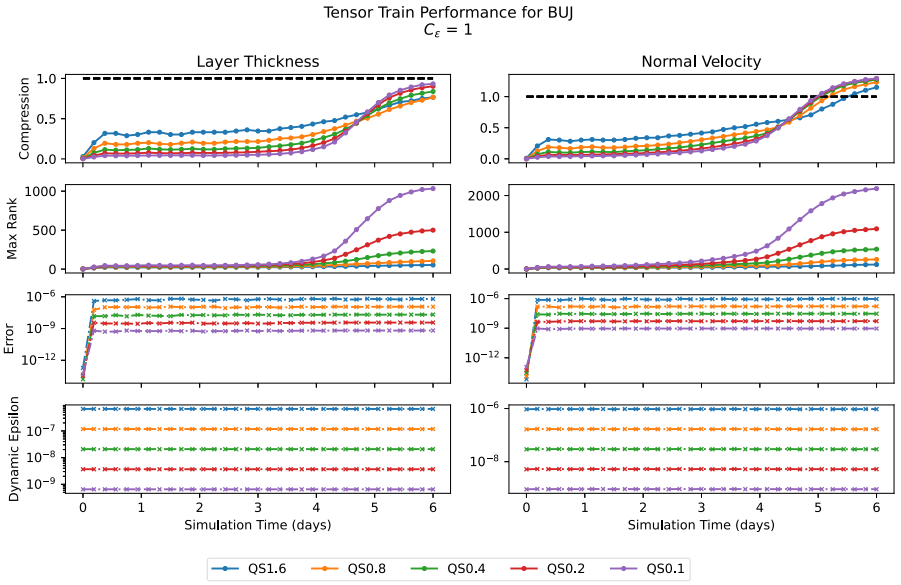


Fig. 13. TT-compression results for BUJ with $C_\varepsilon = 1$.

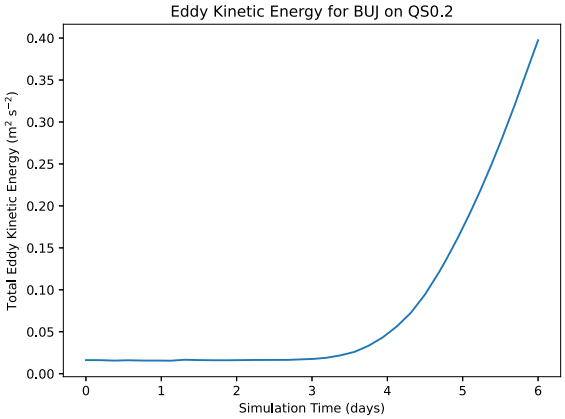


Fig. 14. Total eddy kinetic energy for BUJ on QS0.2.

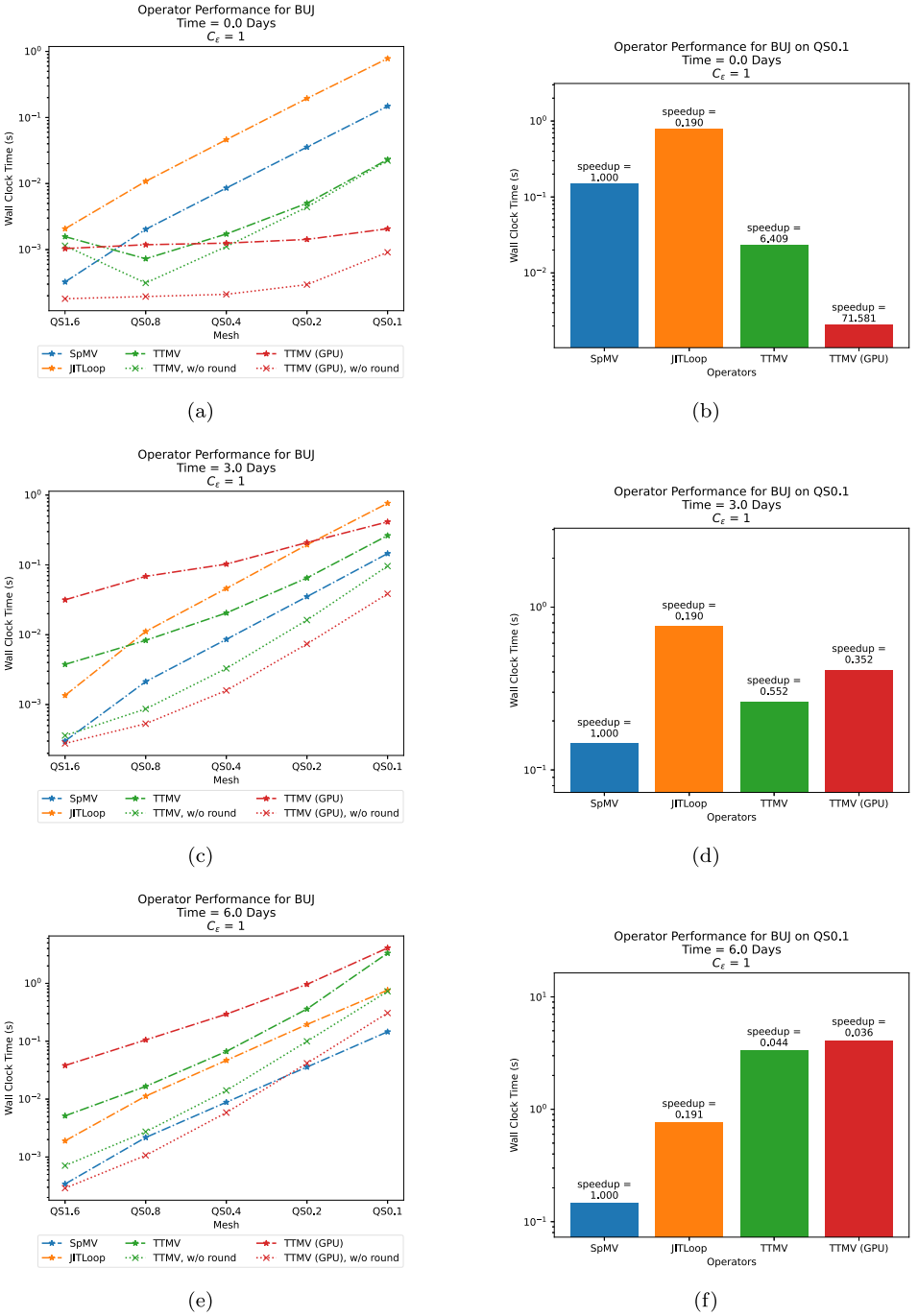


Fig. 15. TT-format operation performance in the BUJ test case.

mature. However, the lack of TT-compression in physically relevant cases like BUJ and WTC5 points to a more fundamental challenge in applying TT-methods to geophysical fluids applications.

The high TT-rank in BUJ and WTC5 are also problematic for the expense that it imposes on the TT-rounding procedure. Taking the final time $t = 50$ days for WTC5 (Fig. 19e and f) as an example, observe the cost incurred by the TT-rounding procedure for TTMV on both CPU and GPU. On the GPU, the cost of the operation without the TT-round is almost as fast as SpMV despite the high anti-compression. Similarly, on the CPU, the TTMV without the TT-round is faster than JITLoop. Accounting for the TT-round, both cases are significancy slower because of the high rank of the result and the $\mathcal{O}(dnr^6)$ cost of rounding [1]. Of course, one does not

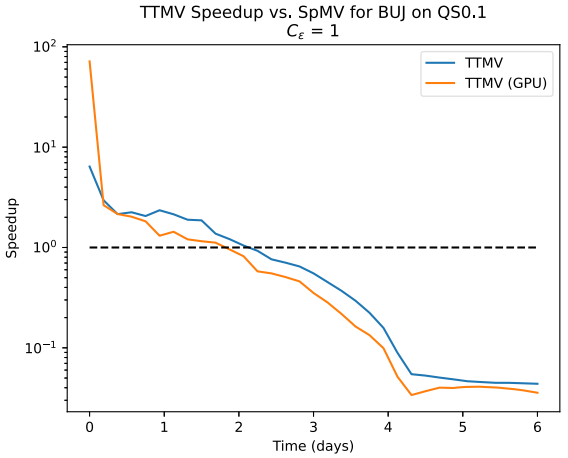


Fig. 16. TTMV speedup versus SpMV in time for the BUJ test case on QS0.1.

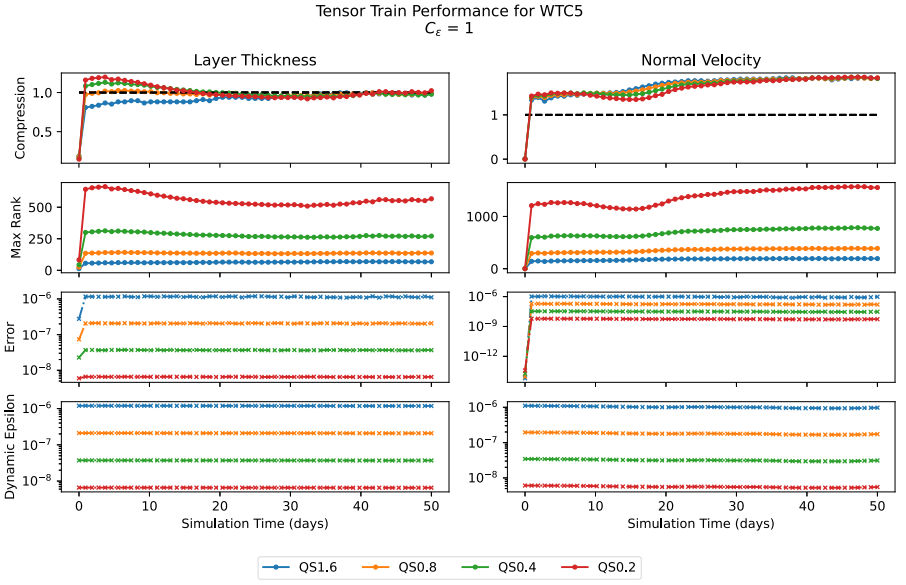


Fig. 17. TT-compression results for WTC5 with $C_\epsilon = 1$.

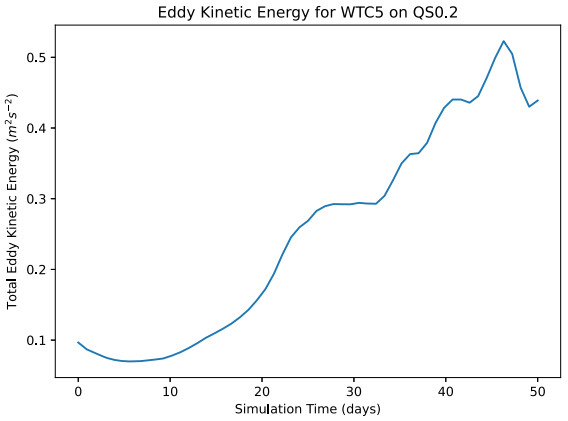


Fig. 18. Total eddy kinetic energy for WTC5 on QS0.2.

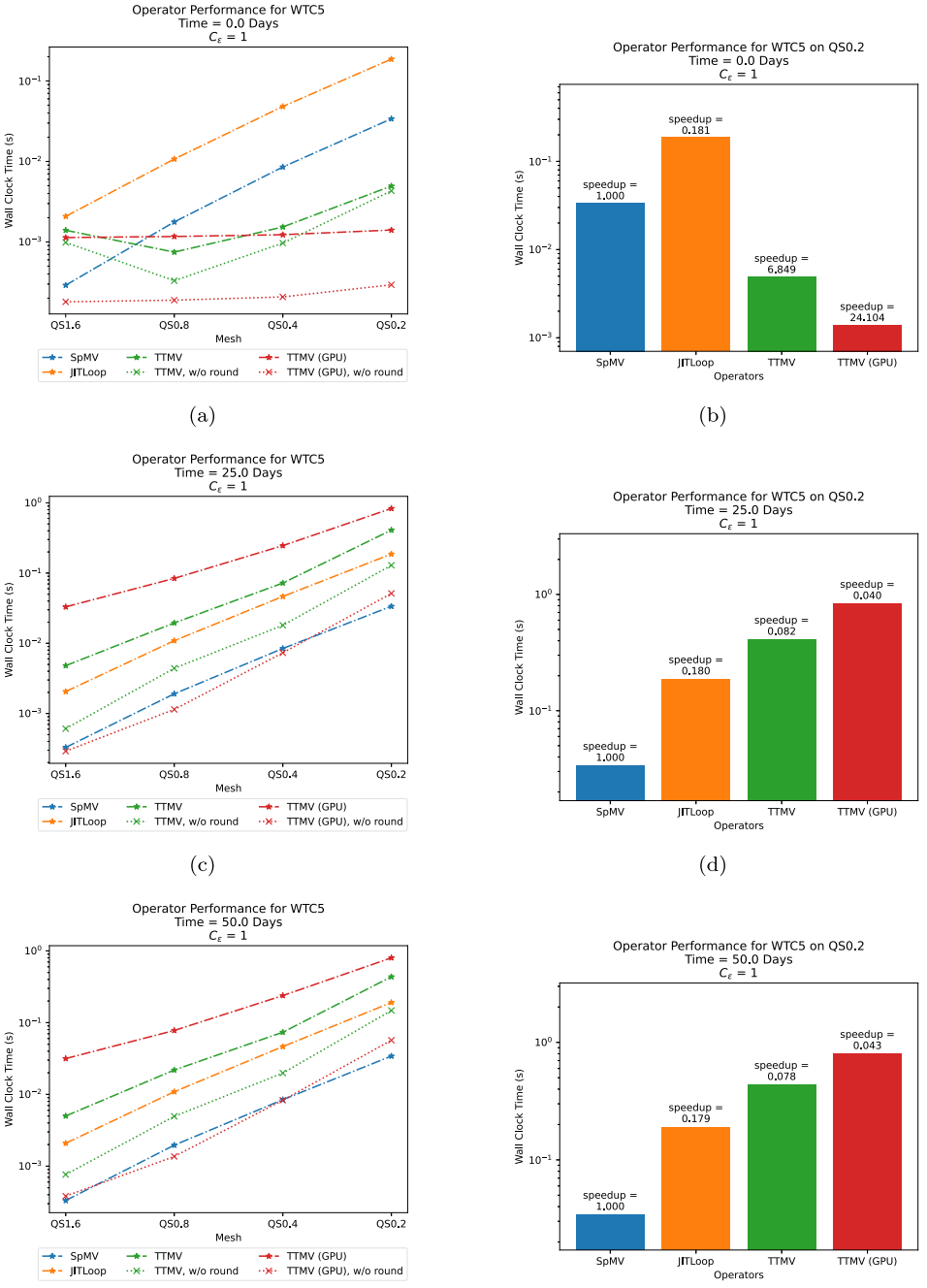


Fig. 19. TT-format operation performance in the WTC5 test case.

necessarily need to round after each and every TT operation, but the longer this is delayed, the more the rank will grow, and the more the $\mathcal{O}(dnr^6)$ cost will effect performance – especially in high-rank cases like BUJ and WTC5.

Finally, we note that because the TT-compression depends on the dimension of the data tensor (Eq. (8)), the compression in all given test cases would likely improve in a higher-dimensional domain such as the full 3-dimensional ocean. Additionally, in a real-world application, cell-centered quantities like the thickness and additional tracers could be bundled into a single higher denominational tensor, potentially furthering the effectiveness of the TT method. This warrants further investigation. However, the poor TT-compression in the velocity field would likely remain problematic, especially as 3-dimensional dynamics further exacerbated the TT-rank growth we see in BUJ and WTC5. Because of the additional computational cost of TT operations, including TT-rounding,

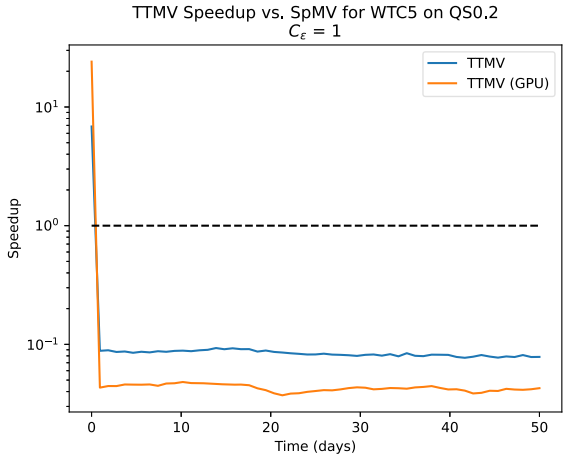


Fig. 20. TTMV speedup versus SpMV in time for the WTC5 test case on QS0.2.

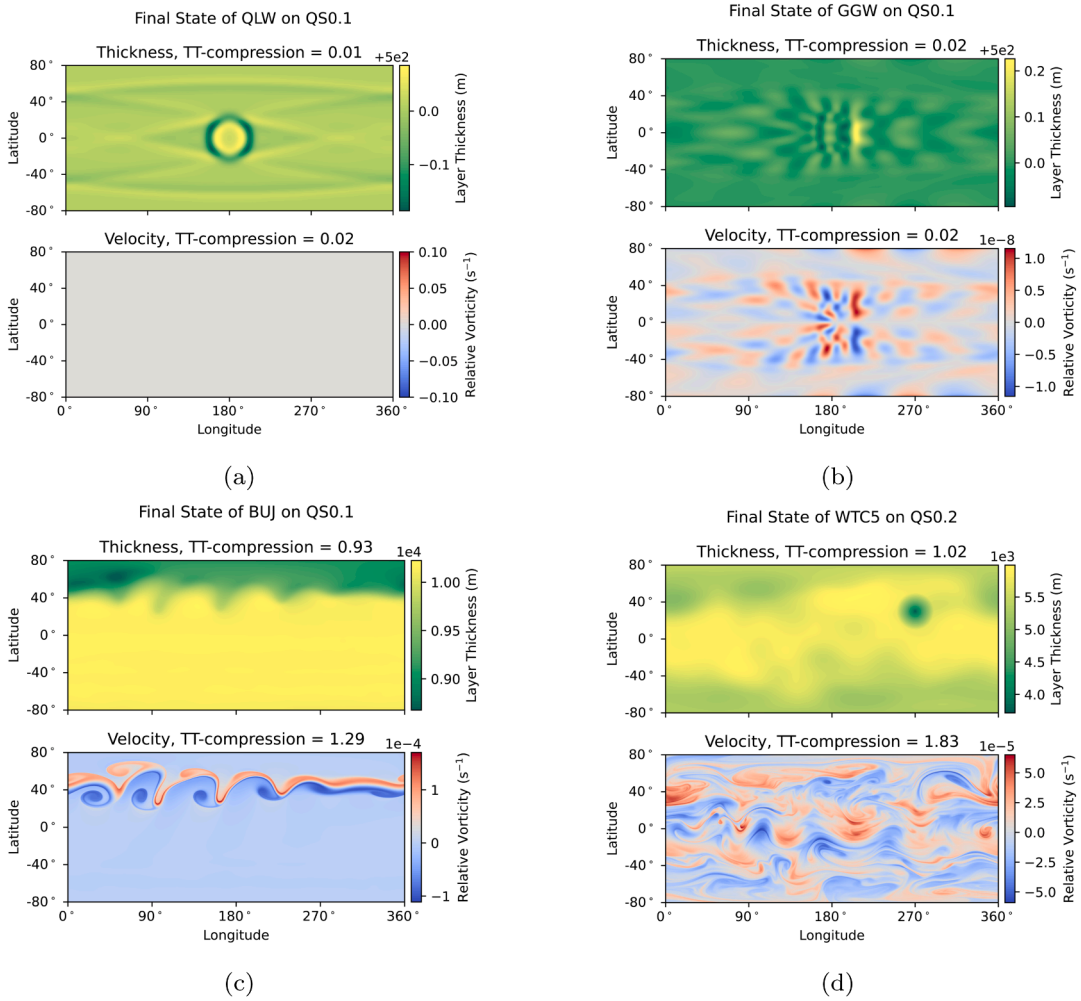


Fig. 21. Snapshots of the final states for each of our four test cases, along with the TT-compression rate achieved with $C_\varepsilon = 1$ (recall that compression greater than one is anti-compression). Note that the velocity plots do not show the velocity itself, but rather the relative vorticity $\nabla \times \mathbf{u}$, which provides a better visualization of the relevant data. The relative vorticity plot for the QLW case shows nothing because the case does not exhibit nonzero vorticity.

the TT-compression must be strong to result in computational speedup (e.g. Figs. 13 and 15); simply having a compression rate less than one is not enough.

5. Conclusion

The results presented in this work show that TT methods can be very effective for simple problems (Figs. 5 and 9), but struggle to efficiently represent more realistic flows (Figs. 13 and 17). In the context of the shallow water equations, Figs. 14 and 18 suggest a correlation between the eddy kinetic energy (i.e. the difference between the velocity state and a smooth mean state) and the TT-compression, with high EKE cases showing poor TT-compression. As such, it is not likely GFD models can achieve meaningful speedups via implementations of full TT-format numerics. As noted in the introduction and in Section 2.2.2, the reason that TT is so effective in the quantum materials space is that it can be shown analytically that all realistic quantum states can be efficiently represented by a TT [11]. Here, we have shown that this is not the case for GFD applications; some states can be efficiently represented with TT, but there exists realistic states that cannot be.

Of course, this does not mean that TT methods, or tensor decomposition methods more generally, cannot be of use of in GFD applications. Since TT is very memory efficient and fast for simpler problems, there is a opportunity to apply TT methods to appropriate subproblems or subsystems within existing GFD codes. For example, one could potentially apply TT only to the gravity wave subsystem within a larger ocean or atmosphere model as this is relatively simple, but computationally expensive because of the rapid motions it entails. Further, it may be that other tensor decomposition methods more general than tensor train could be more effective in this application, particularly for dealing with unstructured grids.

Future work on this application of tensor decomposition methods could include an investigation of the effectiveness of different decomposition formats on different meshes. Additionally, a particularly useful result would be an analytic proof connecting some diagnostic quantity of a given flow to high TT-rank (e.g. if it could be shown that in certain cases, high EKE implies that the flow will have high TT-rank). This would provide a result analogous to that in Orús [11] that showed what subset of flow states can be efficiently represented with TT, allowing researchers across applications to know if TT can be applied to their particular problem in advance.

CRediT authorship contribution statement

Jamie Lilly: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Derek DeSantis:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition, Conceptualization; **Mark R. Petersen:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the U.S. Department of Energy through Los Alamos National Laboratory. Los Alamos National Laboratory is operated by Triad National Security, LLC, for the National Nuclear Security Administration of U.S. Department of Energy (Contract No. 89233218CNA000001). The research presented in this article was supported by the Laboratory Directed Research and Development program of Los Alamos National Laboratory under project numbers 20240782ER (LDRD Seedling) and 20258172CT-IST (ISTI Rapid Response), as well as the Center for Nonlinear Studies under project number 20250614CR-NLS. Computing resources were supplied by the Center for Nonlinear Studies.

Data statement

No AI was used in the production of this work, including the writing of this document and the software used to run simulations and generate figures.

All software and data used in this work is open source and freely available; documentation is generally sparse, but the corresponding author is happy to respond to questions.

sparsett [31,35]:
- GitLab: https://gitlab.com/jamie-lilly/sparsett
- Zenodo: https://doi.org/10.5281/zenodo.21107651

swe-python [27,36]:
- GitLab: https://github.com/jamie-lilly/swe-python/tree/tt-dev
- Zenodo: https://doi.org/10.5281/zenodo.21107861

Simulation data and scripts for data generation, analysis, and plots [37]:
- Zenodo: https://doi.org/10.5281/zenodo.21113938

Data availability

All data/code used in this work is open and freely availible as listed in the Data Statement section of the manuscript.

Appendix A. Tensor train on unstructured grids

The goal of this appendix section is to outline and discuss the challenges in adapting TT methods to unstructured, variable resolution meshes. Our discussion will focus on the types of meshes used by the ocean and atmosphere components of E3SM, which are Voronoi grids [17,18] consisting primarily of hexagons as the primal mesh, with a dual mesh consisting of triangles. The spatial discretization used is a C-grid-type discretization [16] wherein the mass variable is computed on cell centers and the normal component of velocity is computed on cell edges (Fig. A.1).

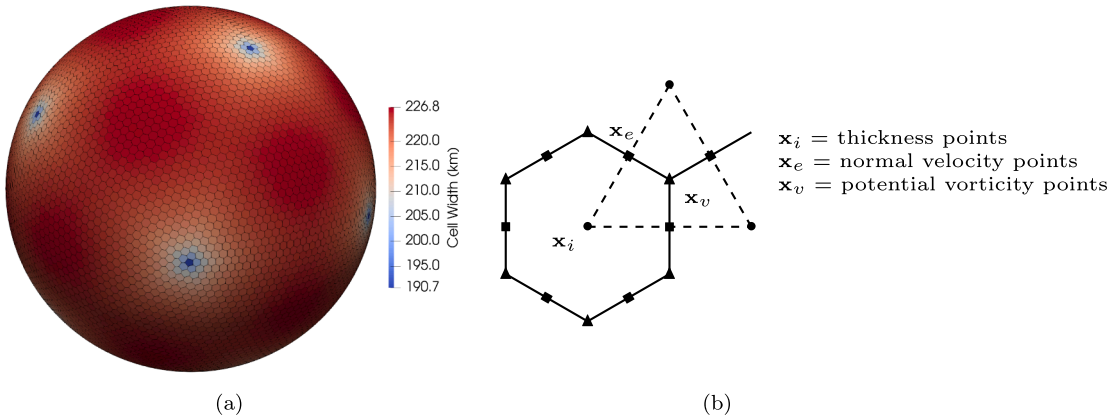


Fig. A.1. Example TRiSK grid from a Voronoi tessellation, where the primal cells are hexagons (and some pentagons) and the dual cells are triangles centered at primal cell vertices.

A.1. Tensorizing an unstructured mesh

In a word, the difficulty is finding a map from a general unstructured mesh to a tensor space $\mathbb{R}^{n_1 \times \cdots \times n_d}$ that allows TT to effectively compress the model state. In the case of the structured meshes as used in the main body of this work (Fig. 4), there is a simple map between the mesh and a tensor space that preserves the spatial locality of data in such a way that the separation of variables performed by taking the TT of the data can exploit that locality to (potentially) find a low-rank representation of the data. That is, the structured 2-dimensional mesh is configured in a way that it can be mapped naturally to a 2-dimensional tensor space. Fig. A.1(a) shows the simplest incarnation of an E3SM-like unstructured Voronoi mesh; there is no obvious mapping from this mesh to a tensor space that is appropriate for decomposition via TT. One can come up with any number of mappings between such a mesh and a tensor space that allows the model data to be given as tensors, but not necessarily one that preserves the data locality, adjacency, and directionality that allows for a low-rank TT representation. This problem only becomes more difficult when we move way from simple meshes like that in Fig. A.1(a), to more realistic meshes as in Fig. A.2.

For now, we will focus on the simpler case from Fig. A.1(a), which we will call ICOS (icosahedral centroidal Voronoi tessellation). In essence, the mapping between such an ICOS mesh and a tensor space can be broken up into two steps; first, ordering the mesh data into a single vector with length equal to the number of data points in the mesh N, and second, folding this vector into a d-dimensional tensor for $d \geq 2$. To ease the discussion, we will refer to these two steps together as the *tensorization* of the model data.

As a simple example, one possible tensorization of an ICOS mesh could be given by first selecting a starting cell (or edge), then visiting each cell (or edge) in a spiral pattern radiating out from the chosen starting element. The vectors of model data could then be sorted according to the order that each element was visited during this spiral traversal. This ordering somewhat preserves data locality, but not data adjacency. The ordered vectors could then be folded according to the prime factorization of N, where a vector of length $N = 2^{\alpha_1}\, 3^{\alpha_2}\, 5^{\alpha_3}\, 7^{\alpha_4}\, \cdots$ is folded into a tensor with shape

$$[\underbrace{2, \cdots, 2}_{\alpha_1 \text{ times}}, \underbrace{3, \cdots, 3}_{\alpha_2 \text{ times}}, \underbrace{5, \cdots, 5}_{\alpha_3 \text{ times}}, \underbrace{7, \cdots, 7}_{\alpha_4 \text{ times}}, \cdots]\,. \tag{A.1}$$

The choice of folding is essentially arbitrary, though is inspired by so-called quantized tensor train (QTT) methods [38,39], which fold a vector of length $N = 2^L$ into shape

$$[\underbrace{2, 2, \cdots, 2}_{L \text{ times}}]\,. \tag{A.2}$$

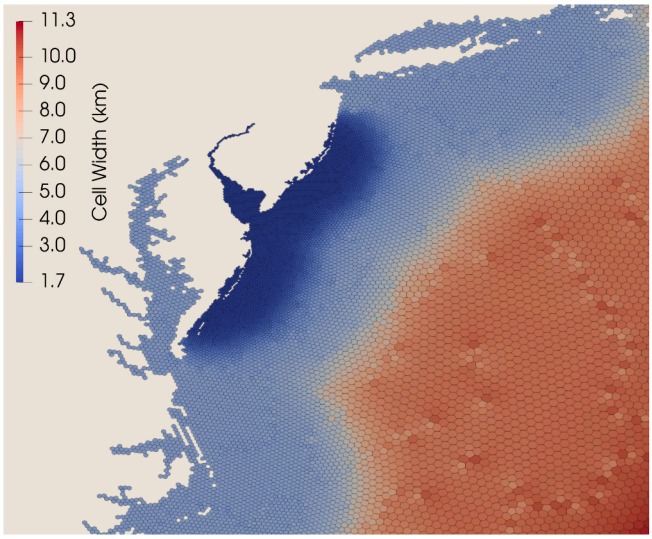


Fig. A.2. A realistic, variable resolution, E3SM ocean mesh zoomed in to Delaware Bay on the east coast of the United States.

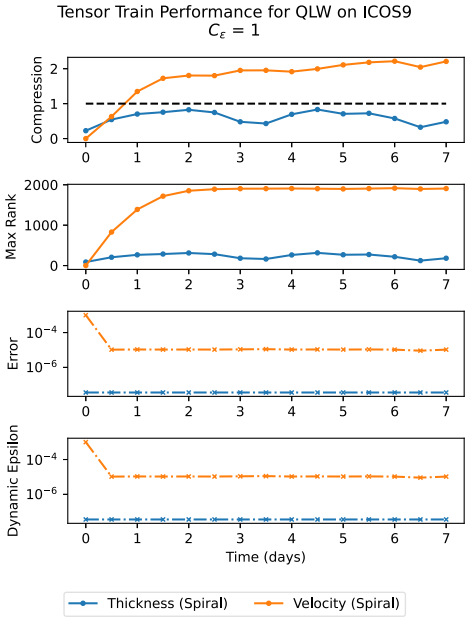


Fig. A.3. TT performance for QLW on a 9th level ICOS mesh (ICOS9). Model data is tensorized using the spiral ordering and QTT-like folding described in Appendix A.1.

The reader might suspect that this is not likely to result in favorable TT performance for a number of reasons, and they would be correct. Fig. A.3 shows the TT compression and error results obtained using these two tensorizations for QLW on an ICOS mesh consisting of 2,621,442 cells (comparable to QS0.2). The layer thickness shows poor compression, and the normal velocity shows very high anti-compression.

The authors experimented with many other tensorization schemes, but none proved effective enough to warrant further study; most showed even worse performance than that in Fig. A.3. Among other strategies, the authors tried applying TT to the default MPAS-O cell order, sorting all mesh cells via a breadth-first search (BFS), and dividing the mesh into many small local regions (similar to a MPI domain decomposition) and sorting those regions via BFS. Another approach could be to re-grid the model data, interpolating it onto a more TT-friendly grid, but this would introduce additional complications related to the fact that our unstructured grids have a nonuniform number of data points along each given spatial axis (i.e. there are more cells at 0° latitude than at 80° latitude). Then, one would need to make decisions about how the re-gridded data was to be loaded into a tensor – if a TT decomposition similar to that used for the QS meshes in the body of this work was wanted, one would need to pad the data tensor with some set of placeholder

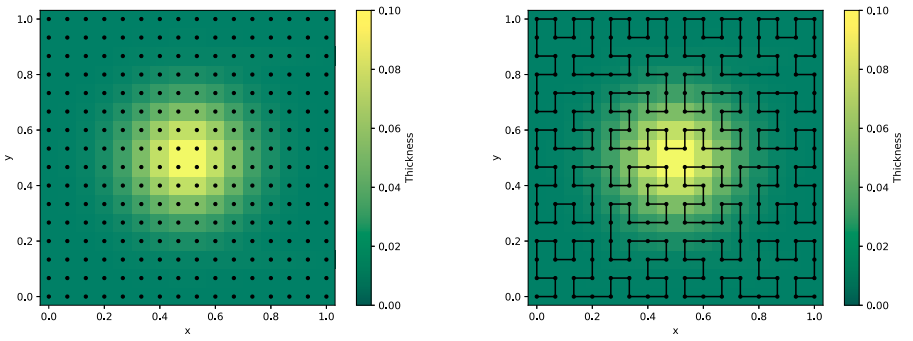


Fig. A.4. A Gaussian bump on a simple 2^ℓ by 2^ℓ grid discretizing $[0, 1] \times [0, 1]$ (a) and an ordering of the grid data given by a Hilbert curve (b).

values so that the tensor was of a uniform size along each axis. In the case of very highly variable resolution meshes, this could lead to a tensor full of mostly placeholder values that could still effect the rank of its TT. Further, we note again that an ICOS mesh gives a greatly simplified tensorization problem; more realistic meshes such as that in Fig. A.2 can only greatly increase the difficulty of the problem.

Another avenue that was briefly explored was the possible use of tensor decomposition methods more general than TT. Also called tensor networks, there are tensor decomposition formats that can be viewed as generalizations of the TT format. For example, the tensor ring (TR) format [21] is similar to TT, but the boundary ranks r_0 and r_d do not have to be equal to one – we only require that $r_0 = r_d$. The resulting tensor network can be viewed as a "ring" of tensors where there are no boundary cores. Another format, generally refereed to as Projected Entangled Pair States (PEPS) [11] generalizes TT to higher dimensions; instead of a one-dimensional array of core tensors, one could have a "matrix" of cores, or even higher dimensional structures. However, with any tensor network, the fundamental problem of finding a map between a mesh and a tensor space to be decomposed remains, and is not necessarily made easier by different network topology.

A.2. Space filling curves

Another possible avenue for tensorizing a general unstructured grid like that in Fig. A.2 could involve space filling curves (SFCs). Given a set of points in $\mathbb{R}^d$ for $d \geq 1$, it is possible to build a curve that visits each point in turn. The simplest example of such a curve is the Hilbert curve (Fig. A.4b).

In the present context, a SFC could be used to provide the needed 1-dimensional ordering our d-dimensional model data – this type of strategy was suggested to the authors in multiple instances. Since this seemed to be a popular and possibly promising idea, we have attempted this strategy on a simplified problem and record the results here. Let ℓ be a positive integer, and consider a uniform 2^ℓ by 2^ℓ grid discretizing $[0, 1] \times [0, 1]$ (the black dots in Fig. A.4a shows such a grid for $\ell = 4$). Let $L = 2\ell$, then there are a total of 2^L grid points which need to be ordered. Then, we assign each of these points a quantity according to

$$g(x, y) = \frac{1}{10} e^{-25[(x-1/2)^2+(y-1/2)^2]} ; \tag{A.3}$$

this is essentially the initial condition for the thickness variable in the QLW test case. The goal here is to see how well this state can be compressed with TT when ordered via a Hilbert curve (Fig. A.4b) and folded into a L-dimensional tensor according to Eq. (A.2). The standard method to order the data is the same used in the body of this work, with the rows of the resulting matrix then concatenated to form a 1-dimensional vector to be folded as in Eq. (A.2) again. Both methods result in tensors of the same dimension and shape, but with a different data layout. The idea is that while the standard method is not easily generalized to unstructured meshes, the Hilbert curve method is.

Table A.1 shows that the TT-compression achieved with the Hilbert curve method is, in each case, an order of magnitude worse than that achieved by the standard method. For large problem sizes the Hilbert curve method compression is still strong, but recall that the state being compressed is very simple. The fact that the maximum TT-rank needed to compress this simple state is high compared to the standard method's maximum rank suggests that this will not extend well to more complicated states. Of course, the use of SFCs for this application is still a largely unexplored avenue; how the above strategy performs when generalized to realistic meshes is unknown and may be the topic of future work. One could apply a general SFC algorithm, such as that given by Sasidharan et al. [40], to an unstructured mesh and perform a study similar to the one done in this work.

The tests in this appendix show that even in the best case scenarios of quad and quasi-uniform meshes (i.e. ICOS), unstructured data ordering prevents TT methods from effective compression. In the more challenging cases of variable-resolution meshes TT would likely perform even worse. This small investigation led to the use of two-dimensional structured grids for the thrust of the study, in order to present the best possible range of results for TT in a spherical shallow water setting.

Table A.1
TT-compression of a simple Gaussian bump for different data orderings using $\epsilon = 10^{-10}$.

Compression		Max Rank
$\ell = 4$; 256 grid points		
Standard	0.313	4
Hilbert curve	2.656	16
$\ell = 8$; 65,536 grid points		
Standard	0.012	8
Hilbert curve	0.419	68
$\ell = 11$; 4,194,304 grid points		
Standard	0.0003	8
Hilbert curve	0.0081	68

Appendix B. Additional results

Here, we provide a collection of additional results for the four test cases from Section 4. TT-compression results are given for $C_\epsilon = 0.1, 10, 100$ for each test case, and additional speedup results are given for BUJ using $C_\epsilon = 10, 100$. In general, we observe that changing C_ϵ does not meaningfully effect the qualitative performance of our TT methods; it does change the TT-compression in each case, but the behavior of the TT-compression curves in each case remains largely the same.

In particular, we draw the readers attention to Figs. B.11 and B.13 , which show the speedup achieved by TTMV versus SpMV for the BUJ case. Compare these to Fig. 16 and observe that increasing C_ϵ doesn't change the overall behavior of the speedup curves. Rather, this only shifts them up on the vertical axis slightly, with the effect that the TTMV methods remain faster than SpMV slightly longer into the simulation before SpMV overtakes them again. This suggests that finding speedup for complex flows like BUJ and WTC5 is not the simple matter of a well-chosen C_ϵ. If we were to increase C_ϵ enough that TTMV was faster than SpMV for the whole of the simulation, the introduced error would simply be too large for the results to be valuable (Figs. B.1–B.16.).

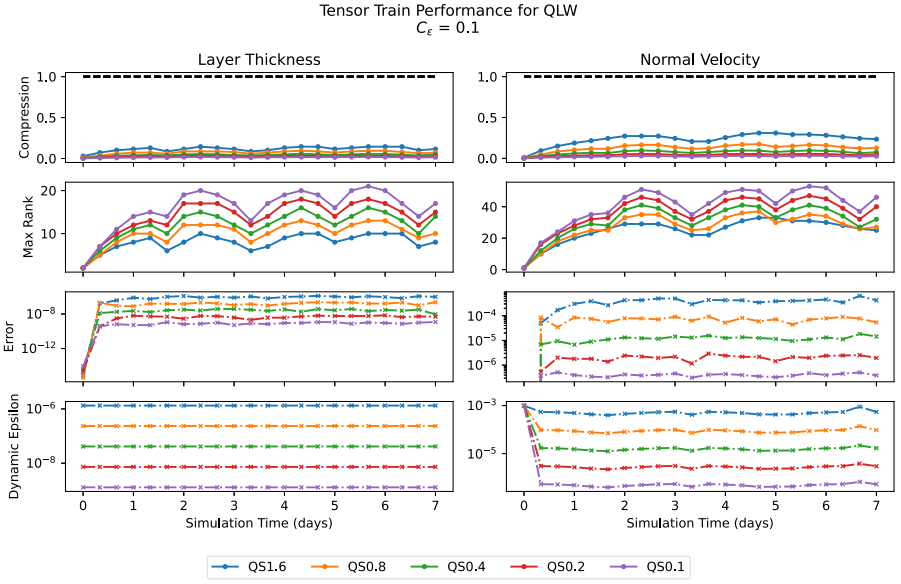


Fig. B.1. TT-compression results for QLW with $C_\epsilon = 0.1$.

B.1. Quasi-linear wave

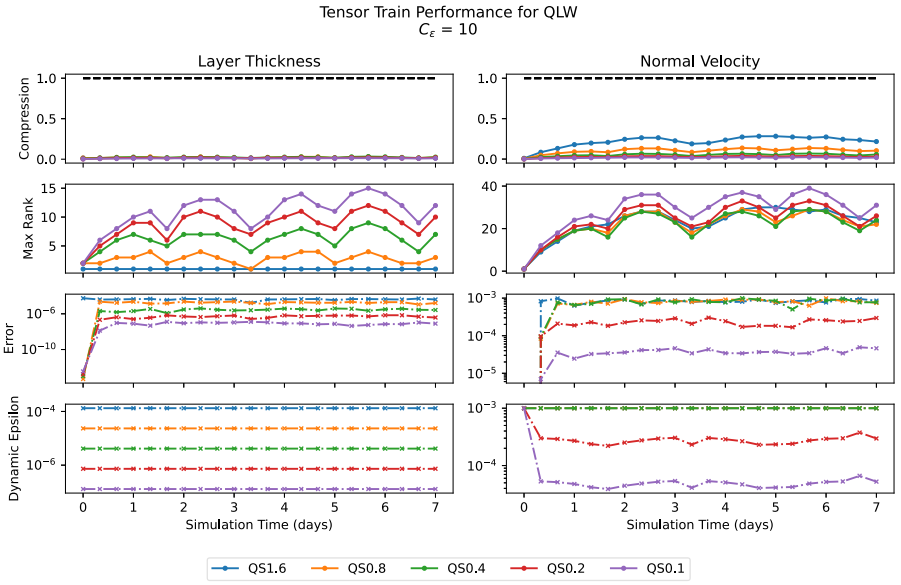


Fig. B.2. TT-compression results for QLW with $C_\epsilon = 10$.

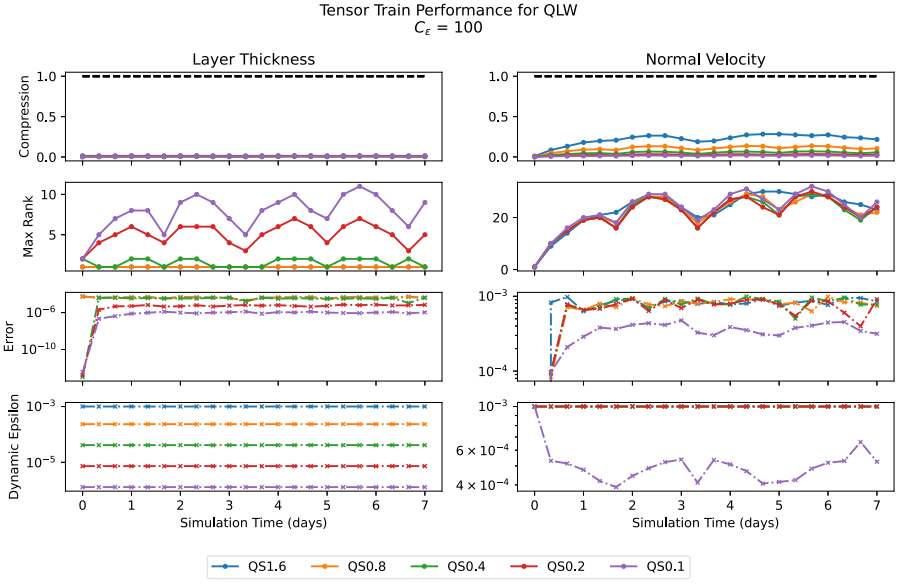


Fig. B.3. TT-compression results for QLW with $C_\epsilon = 100$.

B.2. Geophysical gravity wave

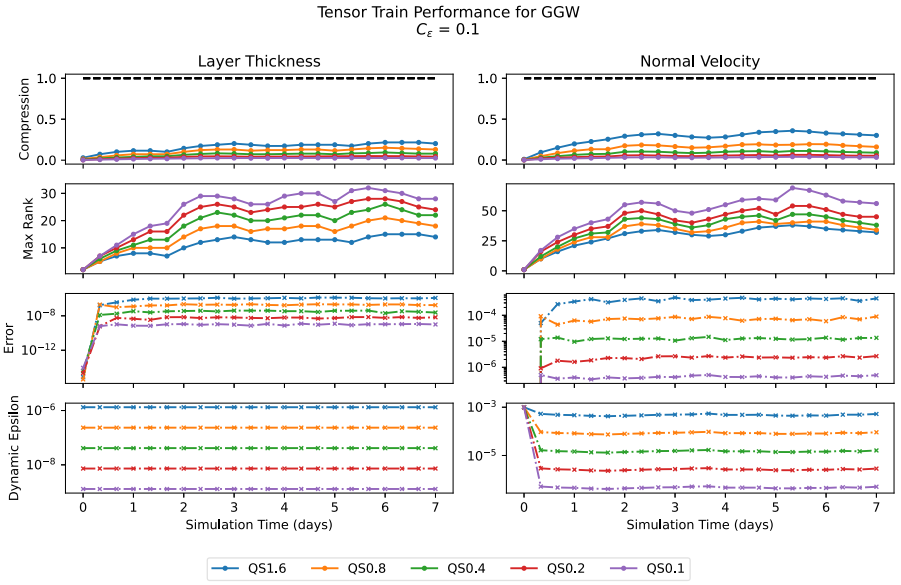


Fig. B.4. TT-compression results for GGW with $C_\epsilon = 0.1$.

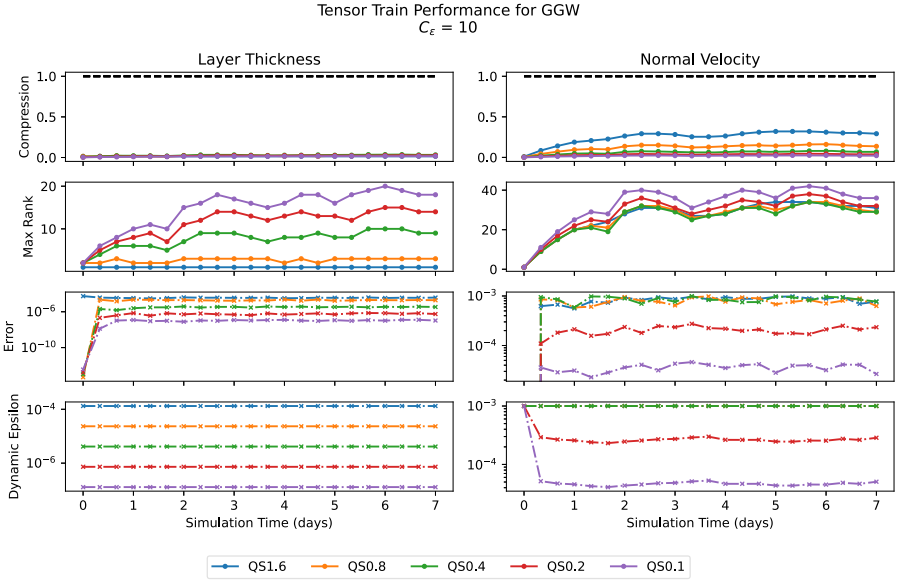


Fig. B.5. TT-compression results for GGW with $C_\epsilon = 10$.

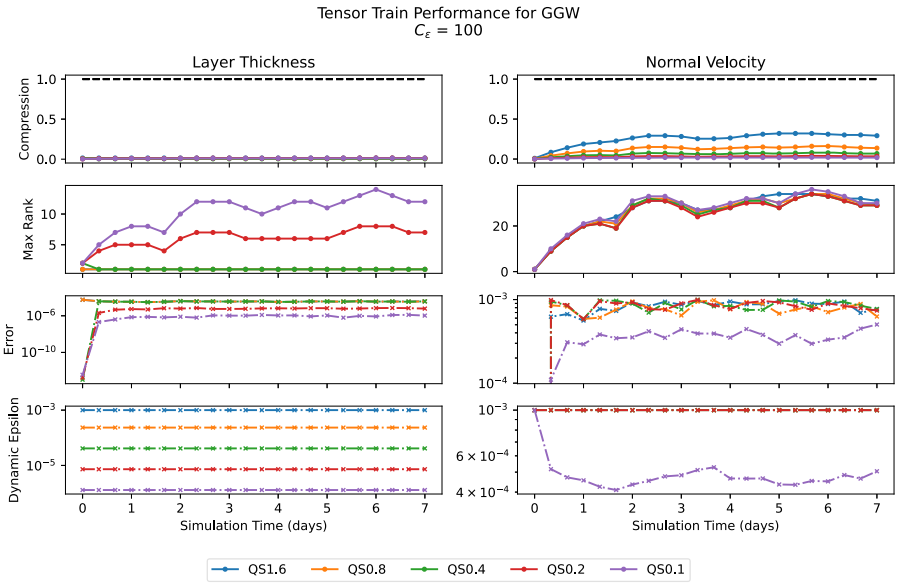


Fig. B.6. TT-compression results for GGW with $C_\epsilon = 100$.

B.3. Barotropically unstable jet

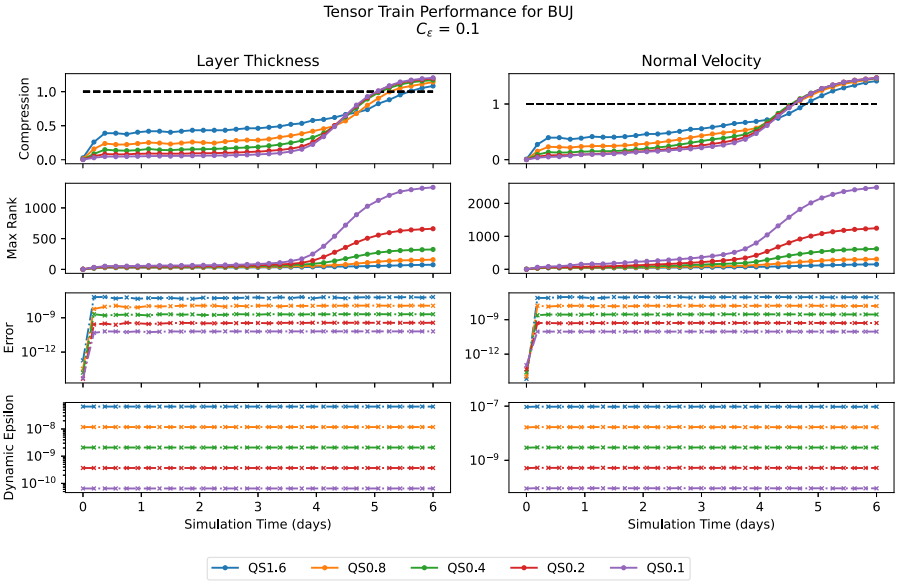


Fig. B.7. TT-compression results for BUJ with $C_\epsilon = 0.1$.

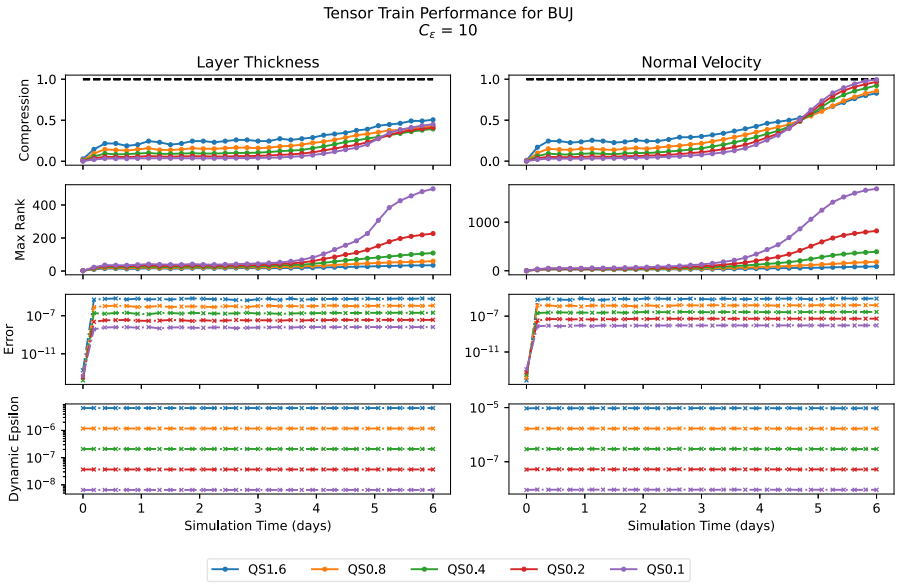


Fig. B.8. TT-compression results for BUJ with $C_\epsilon = 10$.

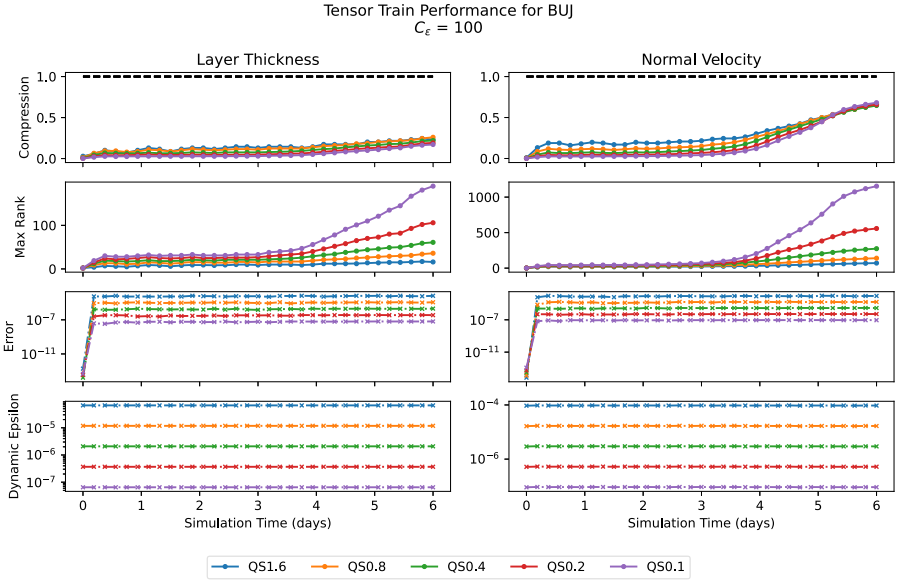
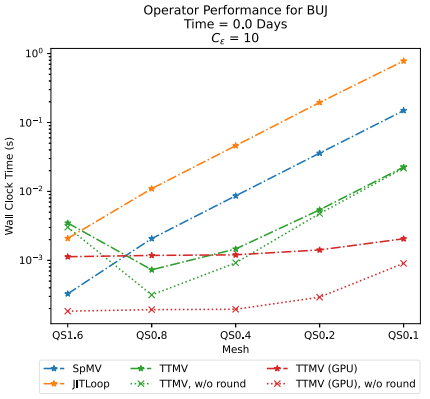
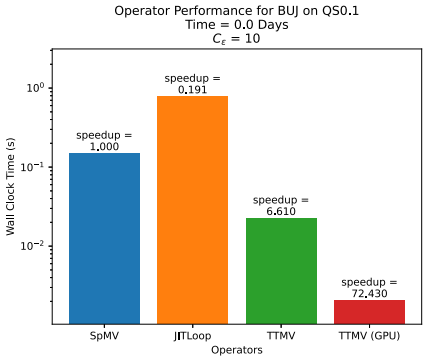


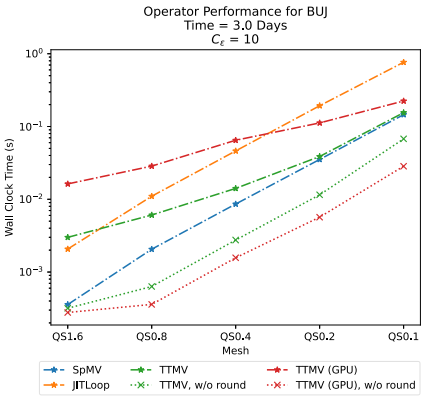
Fig. B.9. TT-compression results for BUJ with $C_\epsilon = 100$.



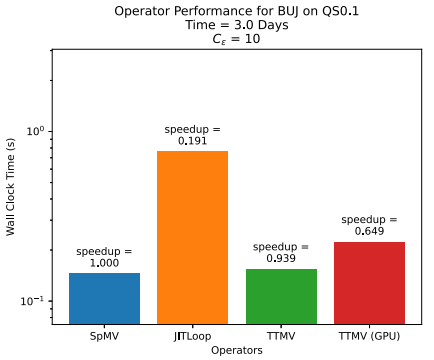
(a)



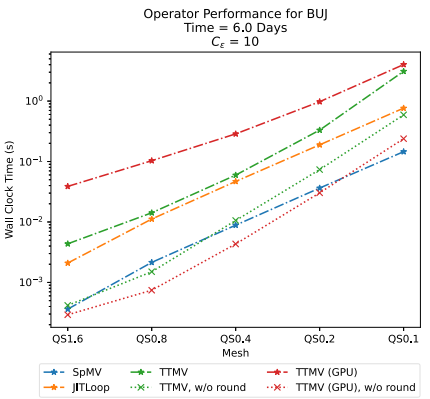
(b)



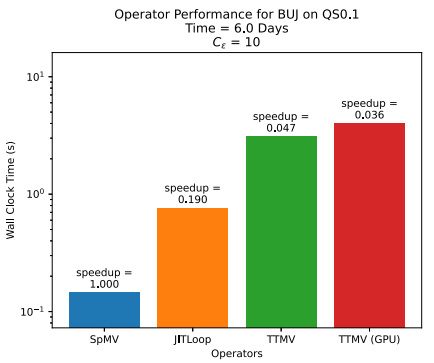
(c)



(d)



(e)



(f)

Fig. B.10. TT-format operation performance in the BUJ test case.

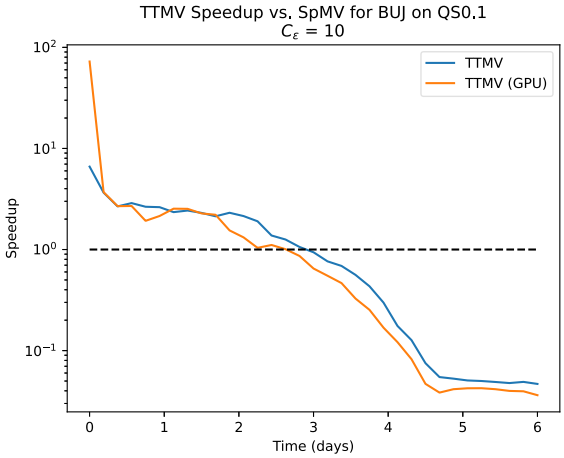
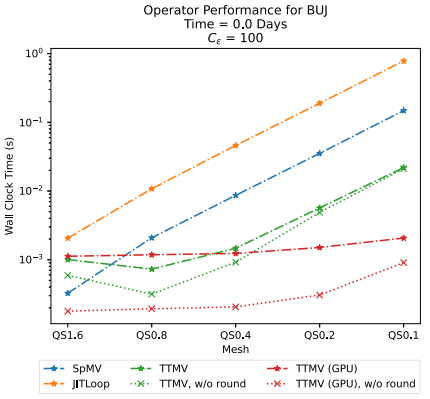
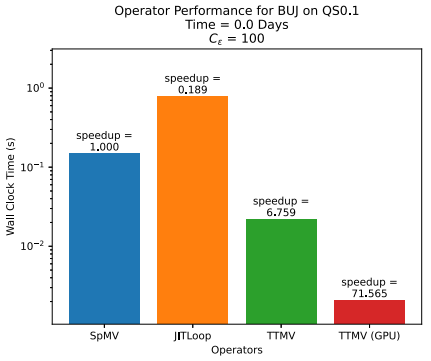


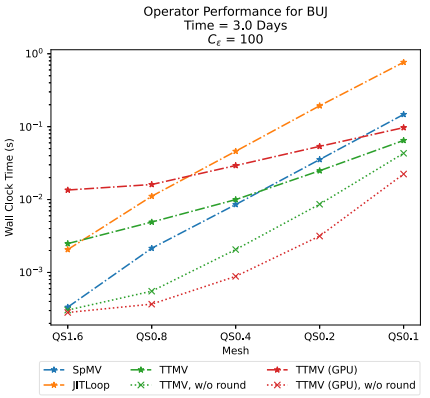
Fig. B.11. TTMV speedup versus SpMV in time for the BUJ test case.



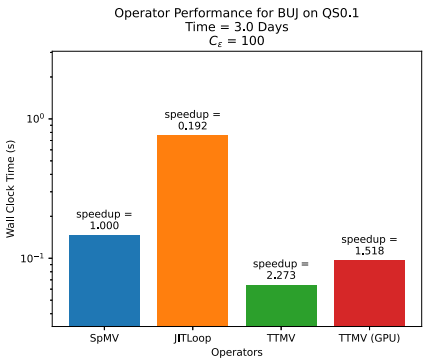
(a)



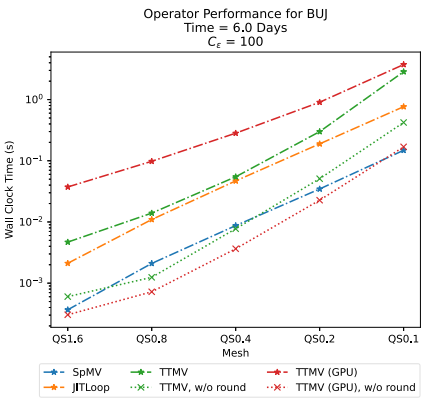
(b)



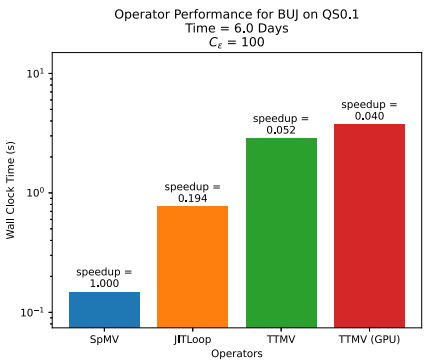
(c)



(d)



(e)



(f)

Fig. B.12. TT-format operation performance in the BUJ test case.

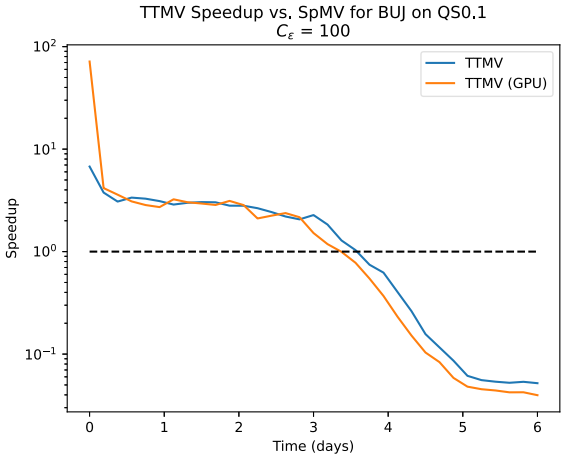


Fig. B.13. TTMV speedup versus SpMV in time for the BUJ test case.

B.4. Williamson test Case 5

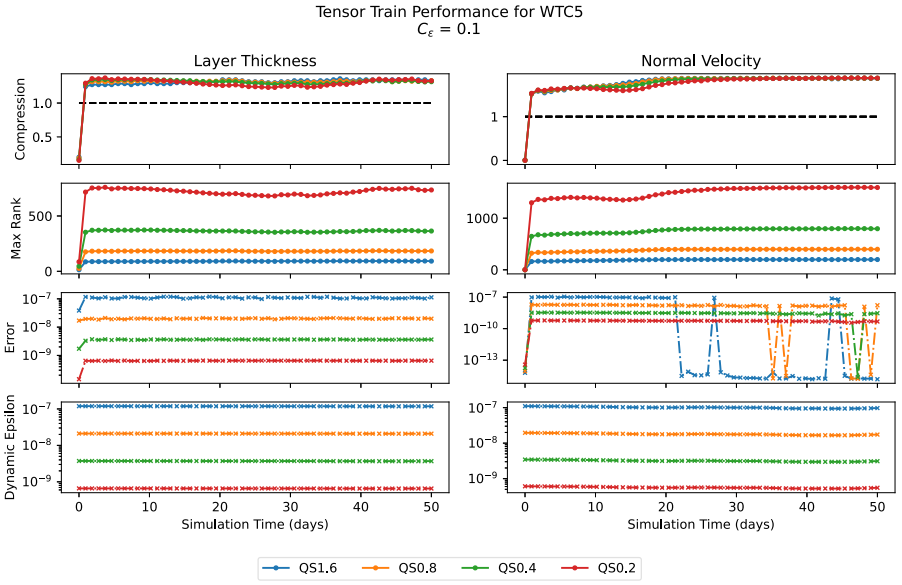


Fig. B.14. TT-compression results for WTC5 with $C_\epsilon = 0.1$.

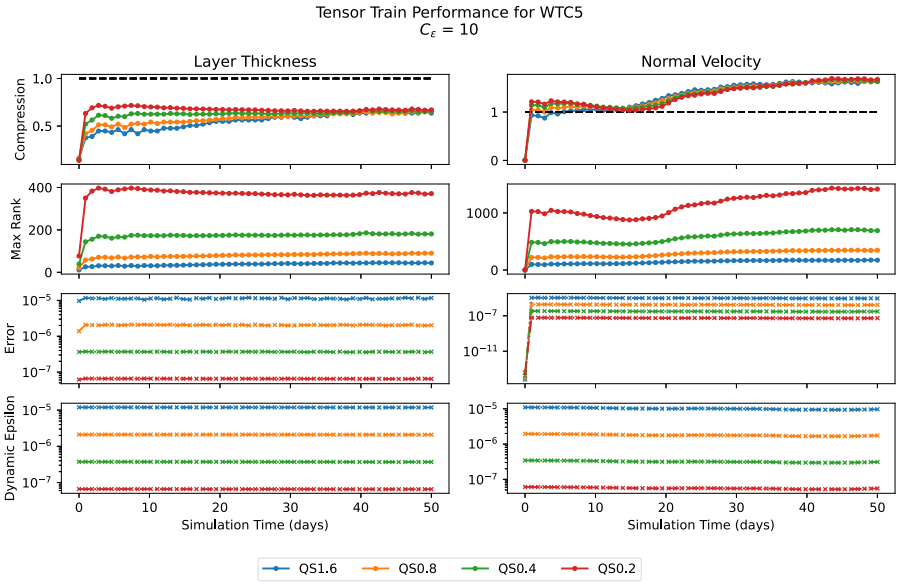


Fig. B.15. TT-compression results for WTC5 with $C_\epsilon = 10$.

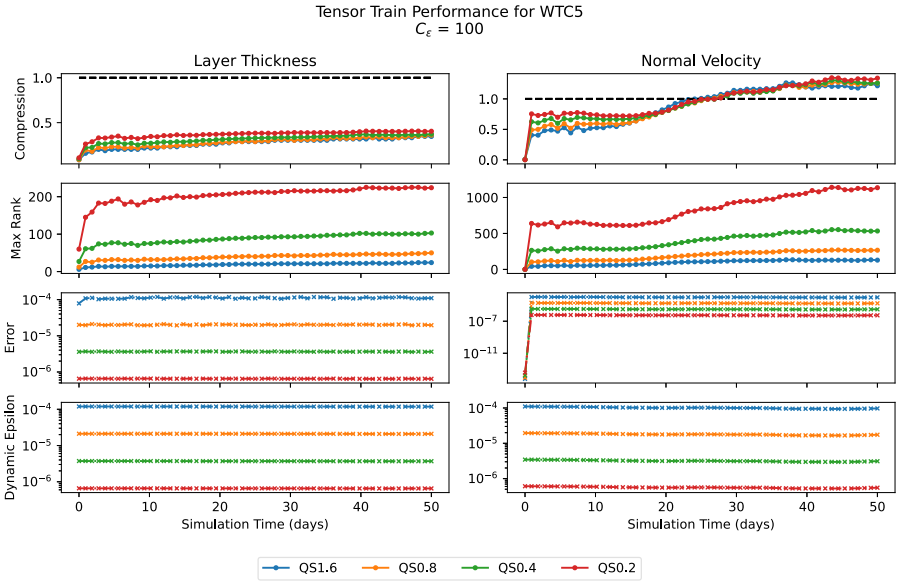


Fig. B.16. TT-compression results for WTC5 with $C_\epsilon = 100$.

References

[1] I.V. Oseledets, Tensor-train decomposition, SIAM J. Scient. Comput. 33 (5) (2011) 2295–2317. https://doi.org/10.1137/090752286
[2] S. Demir, R. Johnson, B.T. Bojko, A.T. Corrigan, D.A. Kessler, P. Guthrey, J. Burmark, S. Irving, Three-dimensional compressible chemically reacting computational fluid dynamics with tensor trains, in: AIAA SCITECH 2024 Forum, AIAA SciTech Forum, American Institute of Aeronautics and Astronautics, 2024, pp. 1–16. https://doi.org/10.2514/6.2024-1337
[3] M. Kiffner, D. Jaksch, Tensor network reduced order models for wall-bounded flows, Phys. Rev. Fluids 8 (2023) 124101. https://doi.org/10.1103/PhysRevFluids.8.124101
[4] T. von Larcher, R. Klein, On identification of self-similar characteristics using the Tensor Train decomposition method with application to channel turbulence flow, Theor. Comput. Fluid Dyn. 33 (2019) 141–159. https://doi.org/10.1007/s00162-019-00485-z.
[5] N. Gourianov, Exploiting the Structure of Turbulence with Tensor Networks, http://purl.org/dc/dcmitype/Text, University of Oxford, 2022.
[6] M.E. Danis, D. Truong, I. Boureima, O. Korobkin, K.Ø. Rasmussen, B.S. Alexandrov, Tensor-train WENO scheme for compressible flows, J. Comput. Phys. 529 (2025) 113891. https://doi.org/10.1016/j.jcp.2025.113891
[7] D.P. Truong, M.I. Ortega, I. Boureima, G. Manzini, K.Ø. Rasmussen, B.S. Alexandrov, Tensor networks for solving the time-independent Boltzmann neutron transport equation, J. Comput. Phys. 507 (2024) 112943.
[8] M.E. Danis, D.P. Truong, D. DeSantis, J. Lilly, M.R. Petersen, K.Ø. Rasmussen, B.S. Alexandrov, High-order tensor-train finite volume methods for shallow water equations, Month. Weather Rev. -1 (aop) (2025). https://doi.org/10.1175/MWR-D-24-0165.1
[9] J.-C. Golaz, L.P. Van Roekel, X. Zheng, A.F. Roberts, J.D. Wolfe, W. Lin, A.M. Bradley, Q. Tang, M.E. Maltrud, R.M. Forsyth, C. Zhang, T. Zhou, K. Zhang, C.S. Zender, M. Wu, H. Wang, A.K. Turner, B. Singh, J.H. Richter, Y. Qin, M.R. Petersen, A. Mametjanov, P.-L. Ma, V.E. Larson, J. Krishna, N.D. Keen, N. Jeffery, E.C. Hunke, W.M. Hannah, O. Guba, B.M. Griffin, Y. Feng, D. Engwirda, A.V. Di Vittorio, C. Dang, L.M. Conlon, C.-C.-J. Chen, M.A. Brunke, G. Bisht, J.J. Benedict, X.S. Asay-Davis, Y. Zhang, M. Zhang, X. Zeng, S. Xie, P.J. Wolfram, T. Vo, M. Veneziani, T.K. Tesfa, S. Sreepathi, A.G. Salinger, J.E.J. Reeves Eyre, M.J. Prather, S. Mahajan, Q. Li, P.W. Jones, R.L. Jacob, G.W. Huebler, X. Huang, B.R. Hillman, B.E. Harrop, J.G. Foucar, Y. Fang, D.S. Comeau, P.M. Caldwell, T. Bartoletti, K. Balaguru, M.A. Taylor, R.B. McCoy, L.R. Leung, D.C. Bader, The DOE E3SM model version 2: overview of the physical model and initial model evaluation, J. Adv. Model. Earth Syst. 14 (12) (2022) e2022MS003156. https://doi.org/10.1029/2022MS003156
[10] T.D. Ringler, J. Thuburn, J.B. Klemp, W.C. Skamarock, A unified approach to energy conservation and potential vorticity dynamics for arbitrarily-structured C-grids, J. Comput. Phys. 229 (9) (2010) 3065–3090. https://doi.org/10.1016/j.jcp.2009.12.007
[11] R. Orús, A practical introduction to tensor networks: matrix product states and projected entangled pair states, Ann. Phys. 349 (2014) 117–158. https://doi.org/10.1016/j.aop.2014.06.013
[12] J. Galewsky, R.K. Scott, L.M. Polvani, An initial-value problem for testing numerical models of the global shallow-water equations, Tellus A Dyn. Meteorol. Oceanogr. 56 (5) (2004) 429. https://doi.org/10.3402/tellusa.v56i5.14436
[13] D.L. Williamson, J.B. Drake, J.J. Hack, R. Jakob, P.N. Swarztrauber, A standard test set for numerical approximations to the shallow water equations in spherical geometry, J. Comput. Phys. 102 (1) (1992) 211–224. https://doi.org/10.1016/S0021-9991(05)80016-6
[14] R.L. Higdon, Numerical modelling of ocean circulation, Acta Numer. 15 (2006) 385–470. https://doi.org/10.1017/S0962492906250013
[15] J.M. Lilly, G. Capodaglio, M.R. Petersen, S.R. Brus, D. Engwirda, R.L. Higdon, Storm surge modeling as an application of local time-stepping in MPAS-ocean, J. Adv. Model. Earth Syst. 15 (1) (2023) e2022MS003327. https://doi.org/10.1029/2022MS003327
[16] AKIO. Arakawa, V.R. Lamb, Computational design of the basic dynamical processes of the UCLA general circulation model, in: JULIUS. Chang (Ed.), Methods in Computational Physics: Advances in Research and Applications, Volume 17 of Circulation Models of the Atmosphere, Elsevier, 1977, pp. 173–265. https://doi.org/10.1016/B978-0-12-460817-7.50009-4
[17] L. Ju, T. Ringler, M. Gunzburger, Voronoi tessellations and their application to climate and global modeling, in: P. Lauritzen, C. Jablonowski, M. Taylor, R. Nair (Eds.), Numerical Techniques for Global Atmospheric Models, Lecture Notes in Computational Science and Engineering, Springer, Berlin, Heidelberg, 2011, pp. 313–342. https://doi.org/10.1007/978-3-642-11640-7_10
[18] A. Okabe, Spatial tessellations, in: International Encyclopedia of Geography, John Wiley & Sons, Ltd, 2017, pp. 1–11. https://doi.org/10.1002/9781118786352.wbieg0601
[19] C. Prévost, P. Chainais, Optimal estimation of the canonical polyadic decomposition from low-rank tensor trains, Signal Process. 234 (2025) 110001. https://doi.org/10.1016/j.sigpro.2025.110001
[20] V. Bhatt, S. Kumar, S. Saini, Tucker decomposition and applications, Mater. Today Proc. 46 (2021) 10787–10792. https://doi.org/10.1016/j.matpr.2021.01.676
[21] P.-L. Wu, X.-L. Zhao, M. Ding, Y.-B. Zheng, L.-B. Cui, T.-Z. Huang, Tensor ring decomposition-based model with interpretable gradient factors regularization for tensor completion, Knowledge-Based Syst. 259 (2023) 110094. https://doi.org/10.1016/j.knosys.2022.110094
[22] L. Xu, L. Cheng, N. Wong, Y.-C. Wu, Tensor train factorization under noisy and incomplete data with automatic rank estimation, Patt. Recogn. 141 (2023) 109650. https://doi.org/10.1016/j.patcog.2023.109650
[23] D. Savostyanov, I. Oseledets, Fast adaptive interpolation of multi-dimensional arrays in tensor train format, in: The 2011 International Workshop on Multidimensional (nD) Systems, 2011, pp. 1–8. https://doi.org/10.1109/nDS.2011.6076873
[24] P. Benner, S. Gugercin, K. Willcox, A survey of projection-based model reduction methods for parametric dynamical systems, SIAM Rev. 57 (4) (2015) 483–531. https://doi.org/10.1137/130932715
[25] S. Chaturantabut, D.C. Sorensen, Nonlinear model reduction via discrete empirical interpolation, SIAM J. Scient. Comput. 32 (5) (2010) 2737–2764. https://doi.org/10.1137/090766498
[26] P.J. Schmid, Dynamic mode decomposition of numerical and experimental data, J. Fluid Mech. 656 (2010) 5–28. https://doi.org/10.1017/S0022112010001217
[27] D. Engwrida, J.M. Lilly, S. Calandrini, swe-python, 2025, (https://github.com/jamie-lilly/swe-python/tree/tt-dev).
[28] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S.J. van der Walt, M. Brett, J. Wilson, K.J. Millman, N. Mayorov, A.R.J. Nelson, E. Jones, R. Kern, E. Larson, C.J. Carey, İ. Polat, Y. Feng, E.W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E.A. Quintero, C.R. Harris, A.M. Archibald, A.H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors, SciPy 1.0: fundamental algorithms for scientific computing in python, Nature Methods 17 (2020) 261–272. https://doi.org/10.1038/s41592-019-0686-2
[29] L. Li, W. Yu, K. Batselier, Faster tensor train decomposition for sparse data, J. Comput. Appl. Math. 405 (2022) 113972. https://doi.org/10.1016/j.cam.2021.113972
[30] H. Abbasi, M. Rocklin, M. Sokół, D. María Léon, D. Tovar, N. Werner, et al., PyData Sparse, 2025, (https://github.com/pydata/sparse).
[31] J. Lilly, sparsett, 2025, (https://gitlab.com/jamie-lilly/sparsett).
[32] S.K. Lam, A. Pitrou, S. Seibert, Numba: a LLVM-based Python JIT compiler, in: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, LLVM '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 1–6. https://doi.org/10.1145/2833157.2833162
[33] xsnm[ion-g-ion], et al., TorchTT, 2025, https://github.com/ion-g-ion/torchTT.
[34] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C.K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M.Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, S. Chintala, PyTorch 2: faster machine learning through dynamic Python Bytecode transformation and graph compilation, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, Volume 2 of ASPLOS '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 929–947. https://doi.org/10.1145/3620665.3640366
[35] J. Lilly, sparsett, 2026, https://doi.org/10.5281/zenodo.21107652
[36] J. Lilly, swe-Python tt-Dev, 2026, https://doi.org/10.5281/zenodo.21107862
[37] J. Lilly, Data and scripts for "viability of tensor train methods for geophysical fluid dynamics", 2026, https://doi.org/10.5281/zenodo.21113939

[38] I.V. Oseledets, Approximation of $2^d\times2^d$ matrices using tensor decomposition, SIAM J. Matrix Anal. Appl. 31 (4) (2010) 2130–2145. https://doi.org/10.1137/090757861

[39] B.N. Khoromskij, O(Dlog N)-quantics approximation of N-d tensors in high-dimensional numerical modeling, Const. Approx. 34 (2) (2011) 257–280. https://doi.org/10.1007/s00365-011-9131-1

[40] A. Sasidharan, J.M. Dennis, M. Snir, A general space-filling curve algorithm for partitioning 2D meshes Proceedings - 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security and 2015 IEEE 12th International Conference on Embedded Software and Systems, HPCC-CSS-ICESS 2015, 17th IEEE International Conference on High Performance Computing and Communications, IEEE 7th International Symposium on Cyberspace Safety and Security and IEEE 12th International Conference on Embedded Software and Systems, HPCC-ICESS-CSS 2015 (2015) 875–879. https://doi.org/10.1109/HPCC-CSS-ICESS.2015.192